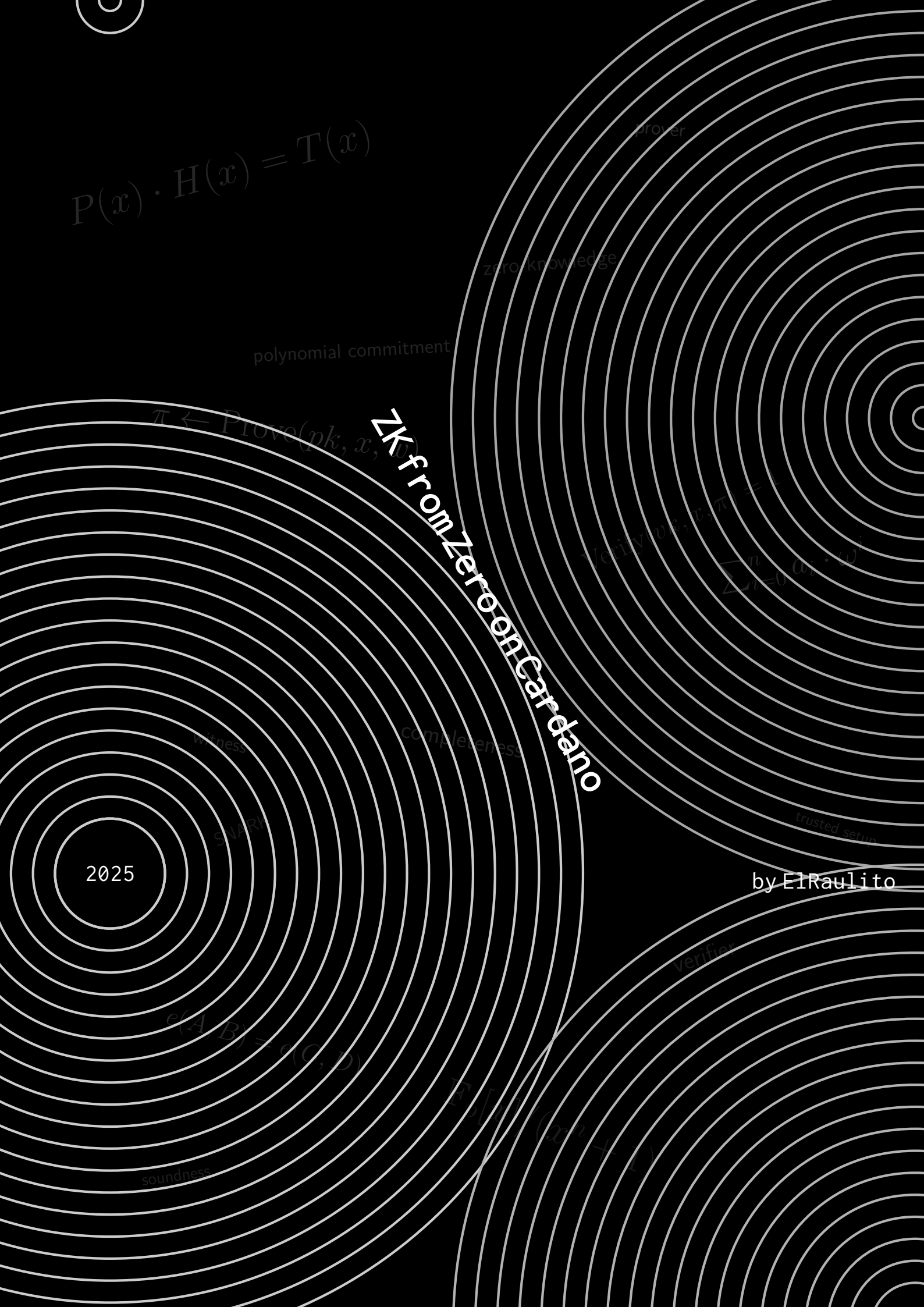



$$P(x) \cdot H(x) = T(x)$$

polynomial commitment

$$\pi \leftarrow \text{Prove}(pk, x, v)$$

ZK from Zero on Cardano

2025

by ElRaulito

soundness

$$e(A, B) = e(C, D)$$

completeness

witness

SNARK

zero knowledge

prover

$$\text{Verify}(vk, v, \pi) = 1 \iff \sum_{i=0}^n a_i \cdot v^i = 0$$

trusted setup

verifier

I FOUNDATIONS OF ZERO-KNOWLEDGE			2
3	1	INTRODUCTION TO ZERO-KNOWLEDGE PROOFS	
	1.1	What are Zero-Knowledge Proofs?	3
	1.2	Core Concepts	4
	1.3	Types of Zero-Knowledge Proof Systems	5
	1.4	ZK on Cardano	5
II ORIGINS AND EVOLUTION			6
	2	THE HISTORY OF ZK	7
	2.1	The Origins of Zero-Knowledge	7
	2.2	From Theory to Practice	7
	2.3	The SNARK Revolution	8
	2.4	Modern ZK: STARKs and Beyond	8
	2.5	ZK Arrives on Cardano	9
III ZK IN WEB3: PRIVACY COINS AND BEYOND			11
	3	ZK IN THE WILD	12
	3.1	Why Web3 Needs ZK	12
	3.2	Zcash	12
	3.3	Monero	14
	3.4	Other ZK Applications in Web3	15
	3.5	Midnight	16
IV ZK FOR COMPRESSION AND PRIVACY			21
	4	PERFORMANCE, PRIVACY AND TRADE-OFFS	22
	4.1	ZK for Data Minimisation and Proof Compression	22
	4.2	Succinctness and Scalability	23
	4.3	Privacy by Design: Applications and Limitations	24

4.4	The Trade-offs: Performance vs. Privacy	25
V	BUILDING ZK ON CARDANO: CIRCUITS AND VERIFICATION	27
5	CIRCOM, SNARKJS, AND AIKEN	28
5.1	The Circuit Pipeline	28
5.2	Writing a Circuit in Circom	28
5.3	The Trusted Setup Ceremony	30
5.4	Generating and Verifying a Proof	33
5.5	Preparing the Verification Key for Cardano	34
5.6	On-Chain Verification in Aiken	35
5.7	Janus Wallet: Putting It All Together	36
VI	YOUR FIRST ZK CONTRACT ON CARDANO	39
6	PASSWORD-LOCKED UTXO WITH GROTH16	40
6.1	What We Are Building	40
6.2	Project Setup	40
6.3	The Circuit	41
6.4	Generating <code>input.json</code>	42
6.5	The Trusted Setup	44
6.6	Generating the Proof	45
6.7	Compressing Points for Cardano	45
6.8	The Groth16 Verifier Library	47
6.9	The Aiken Validator	49
6.10	Testing with Aiken	50
6.11	Off-Chain Code with MeshJS	52
6.12	End-to-End Summary	54
6.13	How This Differs from Janus Wallet	54
VII	OFF-CHAIN LOGIC: PROVERS AND TRANSACTIONS	56

DANO	7	IMPLEMENTING OFF-CHAIN LOGIC FOR ZK IN CAR-	57
	7.1	The On-Chain Part Is the Small Part	57
	7.2	What Software Computes Proofs	58
	7.3	The Pipeline, Step by Step	59
	7.4	Where Should the Prover Run?	62
	7.5	Encoding Proofs as Plutus Data	63
	7.6	Building the Transactions with MeshJS	65
	7.7	What the Transactions Actually Cost	69
	7.8	Generating the Off-Chain Code: <code>aiken-zk</code>	70
	7.9	Operational Concerns	71
	7.10	Failure Modes and How to Recognise Them	73
	7.11	Chapter Summary	74
VIII THE LIMITS OF ZK ON CARDANO			75
76	8	TORNADO CASH ON CARDANO: A CASE STUDY	
	8.1	Scope and Purpose	76
	8.2	What Tornado Cash Actually Does	76
	8.3	Sketching It in eUTxO	78
	8.4	The Withdrawal Circuit	79
	8.5	Wall One: There Is No On-Chain Poseidon	81
	8.6	Wall Two: The Hash Function Dilemma	83
	8.7	Wall Three: eUTxO Contention	84
	8.8	The Smaller Obstacles	86
	8.9	Verdict	87
	8.10	What Would Have to Change	88
	8.11	Chapter Summary	88
IX CLOSING: WHERE ZK ON CARDANO GOES			90

	9	FINAL THOUGHTS AND FUTURE DIRECTIONS	91
	9.1	Where We Started, Where We Are	91
	9.2	The Central Asymmetry	91
	9.3	The Missing Primitives	92
	9.4	The Ecosystem, Honestly Assessed	94
	9.5	What Actually Works Today	94
	9.6	Layer 2 and the Midnight Answer	96
	9.7	Ethics, Regulation, and Building Anyway	96
	9.8	What to Build Next	97
	9.9	Where to Continue	98
	9.10	Closing	98
100	10	SITOGRAPHY: THE ZK ON CARDANO LANDSCAPE	
	10.1	How to Use This Chapter	100
	10.2	This Book	100
	10.3	Cardano ZK Projects	100
	10.4	Circuits and Proving	101
	10.5	Cardano Development	101
	10.6	Cardano Improvement Proposals	102
	10.7	Papers and Foundations	102
	10.8	Reference Implementations Worth Reading	103
	10.9	Where to Ask	103

1. INTRODUCTION TO ZERO-KNOWLEDGE PROOFS

1.1. WHAT ARE ZERO-KNOWLEDGE PROOFS?

All of us have heard about Zero-Knowledge, but what is it exactly? In my experience, many projects have abused this term, using it only to hide something already known, like multi-signature protocols, or centralising data and value behind a web2 backend while claiming ZK legitimacy.

A telling example: a project once claimed to make all transactions anonymous. You would send stablecoins to them, tell them the intended receiver, and they would forward the funds bundled with other transfers so the origin was obscured. There is no ZK in that process; it is simply a centralised mixer.

Where's Waldo and the Hash Lock Analogy

The example that first made ZK click for me is **Where's Waldo** (known as *Where's Wally* outside North America). Imagine you have found Waldo on a busy page but you want to prove to a friend that you know where he is without pointing at him. You could take a very large sheet of paper, cut a small hole over Waldo, and show just his face through the hole. Your friend sees Waldo and is convinced you know the location, without learning anything else about the page.

Using secrets in blockchains was already possible in Bitcoin with **hash-lock scripts**: you choose a password known only to you, hash it, and lock funds to that hash. Anyone who can reveal the pre-image unlocks the coins.

Hash Time-Locked Contracts (HTLCs) are the technology behind cross-chain atomic swaps (used by protocols such as *Fluid Tokens*) and underpins payment routing on the Bitcoin Lightning Network.

The critical limitation of hash locks is that revealing the password burns it: once the pre-image is public, that address is permanently unsafe because anyone now knows the secret tied to it. With ZK you can **prove that you know the password without ever revealing it**. That is the real magic.

ZK as Compression, Not Just Privacy

Most people stop at the privacy angle. ZK is far more useful as a **compression tool**, and this is arguably its most important application in blockchain today.

ZK ROLLUPS (GENERAL CASE).

An L2 network may process thousands of transactions per second. Posting every transaction to L1 would be prohibitively expensive. Instead, the rollup posts only a ZK proof attesting that all those transactions were correctly signed by the users involved.

An L1 smart contract verifies the proof; a single compact check replaces thousands of individual verifications.

Thousands of L2 transactions become one ZK proof on L1.

The on-chain contract only verifies the proof. Nobody can lie about what happened.

THE CARDANO USE CASE.

When you interact with a Cardano DEX today, the contract executes all calculations on-chain: it checks amounts, validates prices, and enforces invariants for *every* order, consuming script budget. With ZK the flow changes completely:

1. You build the transaction and perform all calculations *off-chain*: the amounts to receive, slippage bounds, fee splits.
2. You attach to your transaction a ZK proof of those computations.
3. The DEX contract checks only that the proof is valid. It does not re-execute the math.

Because verification is orders of magnitude cheaper than re-computation, a single transaction can batch hundreds of orders. You see? ZK is being used to **compress computation**, not to hide anything.

1.2. CORE CONCEPTS

Before going further it is worth fixing the three properties that every honest ZK system must satisfy.

- **Completeness.** If the statement is true and the prover is honest, the verifier will always be convinced.
- **Soundness.** If the statement is false, no cheating prover can convince the verifier (except with negligible probability).
- **Zero-Knowledge.** The verifier learns nothing beyond the truth of the statement. The proof reveals no information about the secret witness.

In every ZK protocol there are two parties:

- **Prover:** knows a secret (the *witness*) and wants to convince the verifier without revealing it.
- **Verifier:** checks the proof and decides whether to accept it.

Interactive vs. Non-Interactive Proofs

Early ZK proofs were *interactive*: the verifier sends random challenges and the prover responds, repeating rounds until the verifier is statistically convinced. This is elegant in theory but impractical on a blockchain where there is no live back-and-forth.

The **Fiat-Shamir heuristic** (1986) transforms interactive protocols into *non-interactive* ones: the prover generates the verifier's challenges themselves by hashing the transcript

so far. The resulting proof can be verified by anyone, at any time, without interaction, exactly what on-chain verification requires.

1.3. TYPES OF ZERO-KNOWLEDGE PROOF SYSTEMS

The ZK landscape has exploded in the past decade. Here is a practical map of the main families.

- **zk-SNARKs** (Succinct Non-interactive Arguments of Knowledge): very small proofs and fast verification, but require a trusted setup ceremony. Groth16 and PLONK are the dominant constructions. Used in Zcash, Ethereum’s zkSync and Scroll.
- **zk-STARKs** (Scalable Transparent Arguments of Knowledge): no trusted setup, post-quantum secure, but larger proof sizes. Used by StarkWare and StarkNet.
- **Bulletproofs**: no trusted setup, compact range proofs, but verification is slower and scales linearly with circuit size. Used in Monero and Mimblewimble.
- **PLONK and universal SNARKs**: use a single trusted setup that works for *any* circuit up to a given size, removing the need for per-circuit ceremonies.

System	Trusted Setup	Proof Size	Verification
Groth16	Per-circuit	~200 bytes	Very fast
PLONK	Universal	~500 bytes	Fast
STARKs	None	~45–200 KB	Moderate
Bulletproofs	None	~1 KB	Slow (linear)

1.4. ZK ON CARDANO

Cardano’s **eUTxO model** is well-suited to ZK verification: a script validator receives the proof as part of the redeemer or datum and checks it in a single deterministic execution. Because Cardano contracts run off-chain before submission, the prover can generate the proof locally and attach it; the on-chain script only verifies.

With **Plutus V3**, Cardano gained native cryptographic primitives for the BLS12-381 elliptic curve pairing:

- `bls12_381_G1_add`, `bls12_381_G2_add`: elliptic curve point addition in $\mathbb{G}_1$ and $\mathbb{G}_2$.
- `bls12_381_millerLoop`, `bls12_381_finalVerify`: pairing operations needed to verify Groth16 proofs on-chain.

These primitives are also exposed in **Aiken**’s standard library under `aiken/crypto/bls12_381`, making it practical to write on-chain Groth16 verifiers in a high-level language.

The goal of this book is to start from the very basics (proving to someone that you know a value and can therefore spend a UTxO) and build up to more complex applications: compressing off-chain computation, and eventually constructions similar to privacy protocols (studied here for academic purposes only).

2. THE HISTORY OF ZK

2.1. THE ORIGINS OF ZERO-KNOWLEDGE

Zero-Knowledge proofs were invented in **1985** by Shafi Goldwasser, Silvio Micali, and Charles Rackoff at MIT. Their paper (*The Knowledge Complexity of Interactive Proof Systems*) introduced the concept of an interactive proof where a prover convinces a verifier of the truth of a statement while revealing nothing beyond that truth. The paper won the Gödel Prize in 1993 and is widely considered one of the most influential works in the history of cryptography.

At the time, the result was purely theoretical. The proof systems described required many rounds of interaction, were expensive to compute, and existed only to answer a complexity-theoretic question: *how much “knowledge” does a proof leak?* The answer, in some cases, was: none.

2.2. FROM THEORY TO PRACTICE

The Schnorr Protocol

A landmark step toward practicality came with Claus-Peter Schnorr’s identification scheme (1989). The **Schnorr protocol** is a three-move sigma protocol:

1. The prover commits to a random value.
2. The verifier sends a random challenge.
3. The prover responds with a value that combines the challenge with their secret.

The verifier can check the response without learning the secret. Sigma protocols like this are the ancestors of almost every practical ZK system in use today.

The Fiat-Shamir Heuristic

In 1986, Amos Fiat and Adi Shamir showed how to convert any sigma protocol into a **non-interactive** proof by replacing the verifier’s random challenge with a hash of the prover’s commitment. The resulting proof can be generated offline and verified by anyone; no live interaction needed. This single insight is what makes ZK proofs viable on blockchains.

Schnorr Signatures on Cardano with FROST

Schnorr signatures eventually became a production cryptographic primitive. Today you can use them on Cardano through the **FROST** threshold signature protocol, accessible at <https://www.frost.express/>. FROST allows a group of participants to collaboratively sign a message with a Schnorr signature in a way that is indistinguishable from a single-signer scheme on-chain, with no single party holding the full private key.

2.3. THE SNARK REVOLUTION

Through the 2000s and early 2010s, researchers worked to make ZK proofs efficient enough for real applications. The breakthrough came in 2013–2016 with a family of constructions called **zk-SNARKs**.

TRUSTED SETUP AND GROTH16.

The most efficient SNARK construction is **Groth16** (Jens Groth, 2016). It produces proofs of only ~200 bytes that verify in milliseconds, fast enough for on-chain use. The catch is a *trusted setup ceremony*: a group of participants must generate public parameters, and if any participant keeps their secret “toxic waste”, they could forge proofs. In practice, multi-party ceremonies with hundreds of participants make this risk negligible.

ZCASH AND REAL-WORLD DEPLOYMENT.

Zcash (2016) was the first large-scale deployment of zk-SNARKs. It uses Groth16 to let users make shielded transactions: the blockchain verifies a proof that the sender has enough funds and the transaction is balanced, without revealing amounts or addresses.

UNIVERSAL SETUPS.

Circuit-specific trusted setups are impractical for general use. **PLONK** (2019) and **Marlin** introduced *universal* setups: a single ceremony valid for any circuit up to a given size. This enabled the modern ecosystem of general-purpose zkVMs and zkEVMs.

2.4. MODERN ZK: STARKS AND BEYOND

STARKS.

Eli Ben-Sasson and colleagues at the Technion introduced **zk-STARKs** in 2018. STARKs require *no trusted setup*; all parameters are public and derived from collision-resistant hash functions. They are also believed to be post-quantum secure. The trade-off is proof size: a STARK proof can be tens to hundreds of kilobytes versus a few hundred bytes for a SNARK.

StarkWare built an entire ecosystem around STARKs: *StarkEx* powers major DeFi protocols, and *StarkNet* is a permissionless L2 on Ethereum.

ZK ROLLUPS.

The rollup model took off around 2020–2022. **zkSync**, **Scroll**, and **Polygon zkEVM** all use ZK proofs to compress thousands of Ethereum transactions into a single on-chain verification, achieving Ethereum security with a fraction of the gas cost.

RECENT DEVELOPMENTS.

The frontier is moving fast: *folding schemes* like **Nova** (2021) allow recursive proof composition with near-zero overhead; *lookup arguments* like Plookup dramatically reduce the cost of proving table lookups and bit operations; and *zkVMs* such as RISC Zero let developers write programs in Rust and get a ZK proof of correct execution essentially for free.

2.5. ZK ARRIVES ON CARDANO

Plutus V3 and BLS12-381 Primitives

With the Chang hard fork (2024), Cardano gained **Plutus V3** and a set of native BLS12-381 elliptic curve primitives. These allow on-chain verification of Groth16 proofs without any workarounds. The primitives are available in Aiken’s standard library:

- `aiken/crypto/bls12_381/g1`: operations on the $\mathbb{G}_1$ group
- `aiken/crypto/bls12_381/g2`: operations on the $\mathbb{G}_2$ group
- Pairing operations: `miller_loop` and `final_verify`

See the full API at https://aiken-lang.github.io/stdlib/aiken/crypto/bls12_381/g1.html.

Janus Wallet

The most complete open-source ZK project on Cardano today is **Janus Wallet**, available at:

<https://github.com/leobel/janus-wallet>

Janus demonstrates a working Groth16 verifier on Cardano: a user generates a ZK proof off-chain that they know a secret, and the on-chain Aiken validator verifies the proof and releases the UTxO. It is the closest thing Cardano has to a production-grade ZK smart contract today.

What This Book Covers

The rest of this book is structured as a progressive journey:

1. **Starting from basics:** proving knowledge of a secret value and using a ZK proof to spend a UTxO on Cardano.
2. **Compression:** using off-chain computation with on-chain verification to batch many operations into a single transaction.
3. **Advanced constructions:** privacy-preserving patterns studied for academic purposes. All material here is for educational use. This book does not promote protocols designed to obscure transaction data.

3. ZK IN THE WILD

3.1. WHY WEB3 NEEDS ZK

Blockchain is beautiful precisely because it is transparent, but that same transparency can be turned against you.

In 2022, Tornado Cash was placed on the US government's sanctions list. If your address had interacted with it in any transaction, exchanges could freeze your assets. Worse, someone could *send* you a transaction through Tornado Cash and get your address blacklisted without your consent. Once your address is linked to your real identity, every trade, transfer, and incoming payment becomes traceable. Dirty money from hacks could be sent to you on purpose.

This is why on-chain privacy matters, not to hide wrongdoing, but to protect ordinary users from targeted attacks, surveillance, and financial coercion.

Bitcoin Privacy Practices

Bitcoin has no built-in privacy layer, but the community has developed a set of good practices over the years. I believe these can be replicated straightforwardly on Cardano and are worth stating clearly:

- **Never reuse an address.** Bitcoin and Cardano both support deriving unlimited addresses from a single seed. Each payment should receive a fresh address.
- **Avoid round-number amounts and self-change.** Sending exactly 1 BTC and returning change to yourself are fingerprints that chain analytics tools look for.
- **Use multiple inputs and outputs.** A transaction that mixes several UTXOs from different addresses and fans out to several recipients is harder to trace than a one-to-one transfer.
- **Isolate KYC funds.** If you received ADA from a KYC exchange, keep that UTXO entirely separate from addresses you want to keep private. Mixing them collapses your privacy instantly.

These practices reduce linkability but cannot eliminate it. For stronger guarantees you need a cryptographic construction, which is where ZK enters the picture.

3.2. ZCASH

Zcash gained mainstream attention during the Silk Road era: because Silk Road was a black market where participants needed payment privacy, a coin with strong cryptographic guarantees was attractive. Unfortunately the listing of Zcash was front-run by the US government's take-down and arrest of Ross Ulbricht, the founder of Silk Road, before Zcash could see widespread use there. Ross spent ten years in prison; the last US president promised to pardon him during the campaign and ultimately did; he is now a free man.



The story of Ross Ulbricht is a reminder that financial privacy and personal freedom are deeply connected. Regardless of your view of Silk Road, the case set important legal precedents about online speech, sentencing, and the use of blockchain evidence.

How Zcash Works

Zcash uses **zk-SNARKs** (specifically the Groth16 construction) to enable shielded transactions. Every transaction on Zcash involves two possible address types:

- **Transparent addresses** (t-addresses): behave like Bitcoin addresses; all amounts and addresses are public.
- **Shielded addresses** (z-addresses): balances and transaction details are hidden behind a ZK proof.

When you send ZEC from one shielded address to another, the transaction contains a proof that:

1. You own a note (a UTXO) with sufficient value.
2. The total value in equals the total value out (no inflation).
3. You are not double-spending.

The verifier (the entire network) checks only the proof. Amounts, sender, and receiver remain invisible. The construction relies on a **trusted setup ceremony**, a multi-party computation that generates the public parameters. If every participant in the ceremony destroys their toxic waste, the parameters are safe. Zcash ran the Sapling ceremony in 2018 with hundreds of participants.

Zcash in one sentence: a proof that your transaction is valid, without revealing who you are, how much you sent, or who received it.

The main criticism of Zcash historically has been low adoption of shielded addresses. Most ZEC on exchanges sits in transparent addresses, which undermines the anonymity set. Shielded usage has grown since the Sapling upgrade, but full privacy still requires both sender and receiver to use z-addresses.

3.3. MONERO

Monero takes a different philosophical stance from Zcash: **privacy by default for every transaction**, not opt-in. There are no transparent addresses. Every Monero transaction is private.

Monero achieves this through three complementary mechanisms:

- **Ring Signatures.** When you spend XMR, your transaction is signed by a ring of n possible signers drawn from the UTXO set. An observer cannot tell which ring member is the true sender.
- **Stealth Addresses.** Each transaction generates a one-time destination address derived from the recipient's public key. Only the recipient can link it to their wallet.
- **Confidential Transactions (RingCT / Bulletproofs).** Transaction amounts are hidden using Pedersen commitments. A Bulletproof proves that all amounts are positive and that inputs equal outputs, without revealing the values. Bulletproofs require no trusted setup.

		
Feature	Zcash (ZEC)	Monero (XMR)
Launch Year	2016	2014
Founders	Zooko Wilcox-O'Hearn & Electric Coin Company	Nicolas van Saberhagen (pseudonymous)
Privacy Mechanism	zk-SNARKs (zero-knowledge proofs)	Ring Signatures, Stealth Addresses, RingCT
Privacy Type	Optional (transparent or shielded)	Mandatory (always private)
Consensus Mechanism	Proof-of-Work (Equihash algorithm)	Proof-of-Work (RandomX algorithm)
Total Supply	21 million ZEC	~18.4 million XMR with tail emission
Development Organizations	Electric Coin Company & Zcash Foundation	Monero Core Team & Community Developers
Smart Contract Support	Limited (possible via cross-chain integrations)	None (focused purely on privacy transactions)
Regulatory Accessibility	Higher due to optional transparency	Lower due to enforced anonymity
2025 Market Capitalization	\$10.8 billion USD	\$7.8 billion USD

 Datawallet

Zcash vs. Monero at a glance:

	Zcash	Monero
Privacy model	Opt-in (shielded)	Mandatory (always on)
Proof system	zk-SNARK (Groth16)	Bulletproofs (no ZK circuit)
Trusted setup	Yes (Sapling)	No
Anonymity set	All shielded UTxOs	Ring of n UTxOs
Proof size	~200 bytes	~2 KB

Monero's mandatory privacy means every user contributes to the anonymity set, which is a fundamental advantage. The trade-off is that ring signatures scale linearly, and proving range correctness with Bulletproofs is slower than a SNARK verifier.

3.4. OTHER ZK APPLICATIONS IN WEB3

ZK proofs have spread far beyond privacy coins. Here are the most important application domains and projects.

Mina Protocol

Mina describes itself as the world's lightest blockchain. Instead of storing the full chain history, every node stores only a **constant-size ZK proof** (currently 22 KB) that the entire chain is valid. This proof is recursive: when a new block is produced, the block producer generates a proof that the new state is valid *and* that the previous proof was valid. The chain compresses down to a single proof of everything that ever happened.

This is a stunning result: syncing a Mina node takes seconds regardless of chain age, because you only verify one proof.

Aztec

Aztec is a privacy-first ZK rollup on Ethereum. Its **Noir** language lets developers write circuits in a high-level, Rust-like syntax. Aztec's architecture separates:

- **Private functions:** executed entirely client-side, producing ZK proofs, with no on-chain trace.
- **Public functions:** executed by the sequencer, visible on-chain.

Applications on Aztec can mix both: a private transfer function hides the sender and amount, while a public function updates a visible registry. Noir compiles to ACIR (Abstract Circuit Intermediate Representation), which multiple backends can prove.

Polygon zkEVM

Polygon zkEVM is a ZK rollup that is **EVM-equivalent**: Ethereum smart contracts and tooling work without modification. Every batch of transactions is accompanied by a zk-SNARK proof that the EVM state transition was correct. The L1 Ethereum contract verifies the proof in a single transaction, achieving Ethereum-level security with dramatically lower gas costs per operation.

StarkNet

StarkNet uses **zk-STARKs** (no trusted setup, post-quantum secure) and its own language, Cairo, which compiles to a provable virtual machine. STARKs produce larger proofs than SNARKs but scale better for very large computations and avoid the trusted-setup risk entirely.

3.5. MIDNIGHT

Midnight is a **data-protection blockchain** developed by Input Output Global (IOG) as a partner chain to Cardano. Its goal is to let developers build applications that can *selectively disclose* information while keeping everything else private, using zero-knowledge proofs as the underlying engine.

What Midnight Enables

- **Private Trading.** Decentralized exchanges where order size and execution timing remain hidden from the public ledger.
- **Confidential DeFi.** Private lending, portfolio management, and liquidity pools where balances are shielded.
- **Compliant Identity (ZK-KYC).** Prove that you are over 18, or that you are an accredited investor, without revealing your passport or income documents to every counterparty.
- **Shielded and Unshielded Tokens.** Developers can create tokens that keep balances private by default, or public when the application requires it.
- **Compact Smart Contracts.** Midnight uses a TypeScript-based language for writing privacy-preserving applications, keeping the developer experience accessible.

How Midnight Infrastructure Works

Running on Midnight requires two services working together:

- **Node:** connects to the Midnight network, participates in consensus, and submits transactions.
- **Indexer:** reads data from the node and exposes it through a **GraphQL API**.

The biggest architectural difference from Cardano's REST-based APIs is that the Midnight indexer speaks **GraphQL**. Instead of hitting specific endpoints, you write a query describing exactly what you want, and the API returns only that data. This means fewer round trips and more flexibility, but requires a different mental model if you are used to REST.

Blockfrost hosts both the node and indexer as a managed service across three environments:

Environment	Use
Mainnet	Production
Preview	Testing
Preprod	Testing

The three service endpoints for Mainnet are:

Service	Endpoint
Indexer API	https://midnight-mainnet.blockfrost.io/api/v0/
WebSocket	wss://midnight-mainnet.blockfrost.io/api/v0/ws
Node RPC	https://rpc.midnight-mainnet.blockfrost.io

The Indexer API handles queries for blocks, transactions, contracts, and wallet events. The WebSocket supports real-time subscriptions for block streaming, contract actions, and ledger events. The Node RPC provides direct JSON-RPC access for wallet providers and transaction submission.

Authentication

There are two ways to authenticate with Blockfrost's Midnight endpoints:

- **Header:** pass your API key as `project_id` in the request header (same as Cardano).
- **Query parameter:** append `?project_id=YOUR_PROJECT_ID` to the URL, which is useful for clients that cannot set custom headers, including Midnight.js itself.

When configuring Midnight.js to use Blockfrost, pass the full URL with the project ID embedded:

```
https://midnight-mainnet.blockfrost.io/api/v0/?project_id=YOUR_PROJECT_ID
```

Getting Started with Midnight.js

Midnight.js is the official SDK for building on Midnight. It handles contract deployment, private state management, ZK proof generation, and transaction submission.

PREREQUISITES.

- Node.js 22.x or higher
- Docker (required to run the local proof server)
- A Blockfrost account with a Midnight project (free tier available)

INSTALLATION.

Install the full SDK barrel package:

```
npm install @midnight-ntwrk/midnight-js
```

Or install individual packages if you only need specific capabilities:

```

npm install @midnight-ntwrk/midnight-js-types
npm install @midnight-ntwrk/midnight-js-contracts
npm install @midnight-ntwrk/midnight-js-network-id

```

STEP 1: SET THE NETWORK.

Before any operation, configure which network you are targeting:

```

import { setNetworkId } from '@midnight-ntwrk/midnight-js-network-id';

setNetworkId('preprod'); // or 'mainnet', 'preview'

```

STEP 2: CONFIGURE PROVIDERS.

Midnight.js uses a modular provider model. Each provider handles one concern:

```

import { levelPrivateStateProvider }
  from '@midnight-ntwrk/midnight-js-level-private-state-provider';
import { indexerPublicDataProvider }
  from '@midnight-ntwrk/midnight-js-indexer-public-data-provider';
import { FetchZkConfigProvider }
  from '@midnight-ntwrk/midnight-js-fetch-zk-config-provider';
import { httpClientProofProvider }
  from '@midnight-ntwrk/midnight-js-http-client-proof-provider';

const blockfrostUrl =
  'https://midnight-preprod.blockfrost.io/api/v0'
  + '?project_id=YOUR_PROJECT_ID';

const providers = {
  // Encrypted local storage for your private state
  privateStateProvider: levelPrivateStateProvider({
    storePath: './my-private-state'
  }),
  // GraphQL queries to the Midnight indexer
  publicDataProvider: indexerPublicDataProvider(blockfrostUrl),
  // Fetches ZK artifacts (prover keys, verifier keys)
  zkConfigProvider: new FetchZkConfigProvider('/zk-artifacts'),
  // Sends circuits to the proof server for proving
  proofProvider: httpClientProofProvider('http://localhost:6300')
};

```

The proof server runs locally in Docker. It receives your circuit inputs and returns a ZK proof that can be attached to a transaction. Your private inputs never leave your machine.

STEP 3: DEPLOY A CONTRACT.

```
import { deployContract }
  from '@midnight-ntwrk/midnight-js-contracts';
import { compiledContract } from './your-contract';

const deployed = await deployContract(providers, {
  compiledContract,
  privateStateId: 'my-state',
  initialPrivateState: { counter: 0n }
});

console.log('Contract deployed at:', deployed.deployTxData.contractAddress);
```

STEP 4: CALL A CIRCUIT (PRIVATE TRANSACTION)

```
import { findDeployedContract }
  from '@midnight-ntwrk/midnight-js-contracts';

const contract = await findDeployedContract(providers, {
  contractAddress: deployed.deployTxData.contractAddress,
  compiledContract,
  privateStateId: 'my-state'
});

// Calling a circuit generates a ZK proof locally,
// then submits the transaction to the network.
const tx = await contract.callTx.increment();
await tx.wait();
\end{verbatim}

\paragraph{Step 5 --- Query state.}

\begin{verbatim}
import { getPublicStates }
  from '@midnight-ntwrk/midnight-js-contracts';

const states = await getPublicStates(providers, contractAddress);
console.log('Current public state:', states);
```

Key Concepts to Understand

- **Circuits:** the functions you write in Compact (Midnight's language) that run locally and generate ZK proofs. Think of them as the private business logic.
- **Private state:** data stored on your device, encrypted with AES-256-GCM via LevelDB. It never appears on the blockchain.

- **Ledger state:** the public on-chain state of a contract. Anyone can read it; nobody can fake it.
- **ZK artifacts:** files produced by the Compact compiler, namely prover keys, verifier keys, and the ZKIR (Zero-Knowledge Intermediate Representation). These are loaded at runtime.
- **Transaction flow:** the circuit executes locally, the proof server generates the proof, dust token fees are balanced, and the transaction is broadcast to the network.

Creating a Blockfrost Account

If you already have a Blockfrost account, Midnight support is included at no extra cost; just create a new project and select Midnight Mainnet. If you are new:

1. Sign up at <https://blockfrost.io> (free tier: 50,000 requests per day, no credit card required).
2. Click **Add Project** and select **Midnight Mainnet**.
3. Copy your API key and use it in the URL as shown above.

Midnight requests share your existing Blockfrost rate limit. The GraphQL API also enforces per-query limits on depth, number of fields, complexity, and execution timeout. Requests that exceed these limits return an error describing which limit was hit.

For full SDK documentation see docs.midnight.network/sdks/official/midnight-js and for the Blockfrost-specific configuration see docs.blockfrost.io/midnight/.

4. PERFORMANCE, PRIVACY AND TRADE-OFFS

4.1. ZK FOR DATA MINIMISATION AND PROOF COMPRESSION

A point that is not stressed nearly enough: **ZK is not only about privacy**. In physics, light behaves as both a particle and a wave depending on how you observe it. ZK has a similarly dual nature; it can hide information, but it can also *compress* it. These two properties are completely independent, and in practice the compression use case is arguably more economically important today.

The Core Idea

Suppose an L2 network processes 100,000 transactions in one hour. Each transaction has a sender, a receiver, an amount, and a signature. If you posted all of that data to L1, you would overwhelm it and pay enormous fees.

The ZK alternative: the L2 executes all 100,000 transactions off-chain, then generates a single **ZK proof** attesting that every transaction was correctly signed by the legitimate owner of each wallet. The L1 smart contract verifies only the proof, a task that costs a fixed amount of gas regardless of how many transactions are bundled inside.

100,000 L2 transactions become one ZK proof on L1.
The proof is a few kilobytes. The data it certifies could be gigabytes.

This is called a **ZK Rollup**. Nobody can lie about what happened: if the proof verifies, the state transition is correct. The blockchain does not need to see the individual transactions at all.

Why This Matters for Cardano

Consider a Cardano DEX. Today, every order executes on-chain: the validator checks amounts, validates prices, and enforces invariants for each UTxO, consuming script budget per order. With ZK the flow becomes:

1. The DEX batches many orders off-chain.
2. It generates a ZK proof that all orders were correctly matched and all balances balance.
3. A single on-chain transaction carries only the proof and the net state changes.

The on-chain validator verifies the proof. It does not re-execute the matching logic. This is orders of magnitude cheaper per order and allows throughput that would be impossible otherwise.

The Cost: Proof Generation Time

ZK proofs are not free to generate. The off-chain prover must run the circuit (which is an arithmetic encoding of your computation) over the full witness data. For large circuits this can take seconds to minutes on consumer hardware.

This is the fundamental trade-off of ZK compression: *the prover pays a large upfront computational cost so that the verifier pays almost nothing*. In a rollup setting the prover is a well-resourced sequencer, so the cost is amortised across all the transactions it bundles. For a single-user proof the user's device bears the cost.

4.2. SUCCINCTNESS AND SCALABILITY

The “S” in zk-SNARK stands for **Succinct**. A succinct proof has two properties:

- **Small size.** A Groth16 proof is ~200 bytes regardless of the size of the computation it certifies. A PLONK proof is a few hundred bytes. A STARK proof is larger (~50–200 KB) but still bounded.
- **Fast verification.** Verifying a proof takes milliseconds and a fixed amount of compute; again, regardless of the underlying computation size.

This asymmetry is what makes ZK interesting for blockchains. The blockchain is a slow, expensive, globally-replicated computer. You want to put as little work on it as possible. Succinctness lets you offload arbitrarily large computations and pay only a constant verification cost on-chain.

Recursive Proofs and Proof Aggregation

The compression story gets even more powerful with **recursive proofs**: a proof that another proof is valid. If you can verify a ZK proof inside a ZK circuit, you can build a tree of proofs:

- Each leaf proves a batch of transactions.
- Each internal node proves that its two children proofs are valid.
- The root is a single proof attesting to the entire tree.

This is how Mina compresses its entire blockchain history into 22 KB, and how recursive SNARKs like Nova allow continuous proof accumulation with near-zero overhead per step. The on-chain verifier always sees only one proof, regardless of how deep the tree is.

Scalability in Practice

Approach	Throughput	Security
Pure L1	Limited by block size	Native L1 security
Optimistic Rollup	10×–100×	L1 security + fraud proof delay
ZK Rollup	100×–1000×	L1 security + immediate finality
Recursive ZK	Near unlimited batching	Same as ZK Rollup

ZK Rollups have an important advantage over Optimistic Rollups: there is **no challenge window**. An Optimistic Rollup has to wait days before a withdrawal is final, to give time for fraud proofs. A ZK Rollup is final as soon as the proof verifies, typically within minutes of the batch being posted.

4.3. PRIVACY BY DESIGN: APPLICATIONS AND LIMITATIONS

Building with privacy is genuinely hard, not mathematically, but in terms of user experience.

When you build a standard DApp, you can see everything about your user: their balance, their history, their open positions. You can use this data to personalise the interface, warn them about risks, guide them through deposit flows, and show them relevant information. The transparency of the blockchain is, paradoxically, a *UX feature* for most applications.

The moment you introduce privacy, that transparency disappears. You can no longer read the user's balance. You cannot verify their eligibility client-side without the user handing you a proof. The interface has to work without the information it used to take for granted.

Tornado Cash as an Academic Case Study

This book does not promote or endorse the use of Tornado Cash or any sanctioned platform. The following is a purely technical and UX analysis for educational purposes.

Tornado Cash is the clearest example of where ZK privacy meets UX friction. The flow for a user was:

1. Deposit funds into the contract. Receive a **note**, a file containing a cryptographic secret that proves ownership of the deposit.
2. Store the note securely (it is the only way to redeem; lose it and the funds are gone).
3. Later, from a different address with no connection to the original, present the note and a ZK proof that you know the secret. The contract releases the funds to a fresh address.

The cryptography is elegant. The UX is a disaster for non-technical users. Downloading

a file, storing it safely, transferring it across devices, and then running a separate withdrawal flow is completely alien to anyone who has not used a command-line wallet.

The cypherpunk community loves this kind of design; you are in full control, you hold your own keys and notes, nobody can censor you. But if the target audience is mass adoption, this is a hard wall. Most users will make a mistake, lose the note, or panic and try to contact support that does not exist.

Designing Private Interfaces

The lesson is not that privacy and good UX are incompatible; it is that building private applications requires re-thinking the interface from scratch rather than bolting ZK onto an existing transparent design. Some strategies that help:

- **Wallet-managed private state.** Instead of the user handling notes manually, the wallet generates and stores them automatically. Midnight's architecture does exactly this: private state lives encrypted in your local wallet, and the SDK manages proof generation behind the scenes.
- **Progressive disclosure.** Show the user only what they need to see at each step. The complexity of ZK proofs is an implementation detail; it does not belong in the UI.
- **Sensible defaults.** Make the private path the default rather than opt-in. Monero's design (always private) achieves better anonymity sets than Zcash's (opt-in) precisely because users do not have to make a choice.

4.4. THE TRADE-OFFS: PERFORMANCE VS. PRIVACY

Introducing ZK privacy fundamentally changes the computational model of your application, and it is worth being explicit about where the costs land.

Off-Chain Computation

Ethereum popularised an on-chain computation model: all logic executes on the network, results are public and globally verifiable, and the node is the source of truth. Privacy with ZK inverts this: **all meaningful computation happens off-chain**, and the on-chain component verifies only a proof that the computation was done correctly.

This shift has real consequences:

- **User devices bear the proving cost.** Generating a ZK proof can take seconds to minutes on a mobile device. This is improving rapidly with hardware acceleration and more efficient proof systems, but it is not free.
- **Application logic must be expressed as a circuit.** Not all computations are equally easy to express in arithmetic circuit form. Division, conditionals, and floating-point operations all require careful encoding. Languages like Noir, Circom, and Compact (Midnight) exist to make this easier, but there is still a learning curve.
- **Debugging is harder.** When a proof fails to verify, the error is usually a constraint violation deep inside the circuit, not a friendly stack trace. Tooling is improving but lags behind standard development environments.

Trusting a Proving Server

One workaround for slow on-device proving is to offload it to a **centralised proof server**. The user sends their inputs to the server, the server generates the proof, and returns it. This is faster, but it introduces a critical trust assumption: you are now telling a third party the very information you are trying to keep private.

The rule: if the proving happens on a server you do not control, and that server can see your private inputs, then you do not have ZK privacy; you have *server-side secrecy*, which is a much weaker guarantee.

The correct architecture is to run the proof server locally. Midnight does this via a local Docker container: your private inputs go to a process on your own machine and never leave it. The tradeoff is that the user must run Docker, which is again a UX challenge.

Summary of Trade-offs

Choice	Benefit	Cost
On-device proving	True privacy	Slow, hardware-dependent
Centralised proof server	Fast, seamless UX	Privacy broken (trust the server)
ZK for compression only	Scalability, no UX friction	Complexity, proving time
Always-on privacy (Monero)	Large anonymity set	No opt-out, regulatory friction
Opt-in privacy (Zcash)	Regulatory flexibility	Small anonymity set

That is enough theory. From the next chapter onwards we will stop talking about ZK and start *building* it: our first ZK smart contract on Cardano using Plutus.

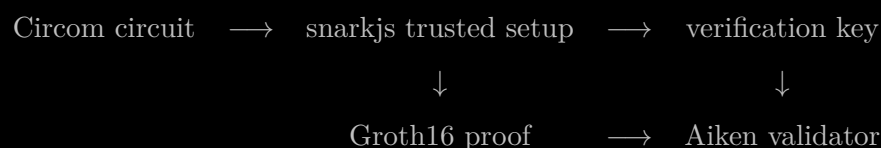
5. CIRCOM, SNARKJS, AND AIKEN

5.1. THE CIRCUIT PIPELINE

Writing a ZK smart contract on Cardano is not a single-language problem. It involves a chain of tools that each play a different role:

1. **Circom**: a domain-specific language for describing arithmetic circuits.
2. **snarkjs**: a JavaScript library that runs the trusted setup ceremony, compiles witness calculations, and generates / verifies Groth16 proofs.
3. **Aiken**: the on-chain language where the Cardano validator lives and where the ZK proof is verified against the circuit's public verification key.

The high-level flow is:



The circuit defines the *rules*. The trusted setup ceremony produces the public parameters that let anyone prove and verify statements about those rules. The proof is generated off-chain by the user. The Aiken validator checks the proof on-chain, a single, fast pairing check using the BLS12-381 primitives introduced in Plutus V3.

5.2. WRITING A CIRCUIT IN CIRCOM

Circom is a language for expressing computations as **arithmetic circuits**, networks of addition and multiplication gates over a finite field. Every statement you want to prove must be reducible to a set of polynomial constraints called an **R1CS** (Rank-1 Constraint System).

Installing the Tools

Install snarkjs globally:

```
npm install -g snarkjs
```

Install Circom by following the official guide at docs.circom.io/getting-started/installation. On Linux/macOS this compiles from Rust source:

```
git clone https://github.com/iden3/circom.git
cd circom
cargo build --release
cargo install --path circom
```


Install the circomlib circuit library (contains Poseidon and other primitives):

```
npm install -D circomlib
```

The Janus Wallet Authentication Circuit

Janus Wallet¹ is the most complete open-source ZK project on Cardano. It uses a circuit that proves a user knows their password without ever sending it on-chain. The circuit is built on the **Poseidon** hash function, a ZK-friendly hash designed to be cheap to prove inside arithmetic circuits (unlike SHA-256, which is extremely expensive to circuit-encode).

The circuit has five **public** inputs (visible on-chain) and one **private** input (the secret that never leaves the user's device):

```
pragma circom 2.1.6;

include "../../node_modules/circomlib/circuits/poseidon.circom";

template Authenticate() {
    signal input userId;           // Public: username identifier
    signal input credentialHash;   // Public: Poseidon(pwd, userId)
    signal input challenge;        // Public: nonce from the smart contract
    signal input challengeFlag;    // Public: overflow flag for challenge
    signal input circuitHash;      // Public: Poseidon(pwd, userId, credentialHash,
                                //                                     challenge, challengeFlag)
    signal input pwd;              // Private: user password

    // Step 1: verify credentialHash = Poseidon(pwd, userId)
    component p = Poseidon(2);
    p.inputs[0] <== pwd;
    p.inputs[1] <== userId;
    credentialHash == p.out;

    // Step 2: verify circuitHash = Poseidon(pwd, userId, credentialHash,
    //                                     challenge, challengeFlag)
    component poseidon = Poseidon(5);
    poseidon.inputs[0] <== pwd;
    poseidon.inputs[1] <== userId;
    poseidon.inputs[2] <== credentialHash;
    poseidon.inputs[3] <== challenge;
    poseidon.inputs[4] <== challengeFlag;
    circuitHash == poseidon.out;
}

component main {public [userId, credentialHash, challenge,
                        challengeFlag, circuitHash]} = Authenticate();
```

The `==` operator asserts a constraint: if the constraint is violated, no valid proof can be produced. The `<==` operator assigns and constrains in one step.

¹<https://github.com/leobel/janus-wallet>

The `challenge` and `challengeFlag` fields are critical for replay protection. The smart contract changes the challenge after every successful spend, so a proof generated for one transaction cannot be reused in another.

Compiling the Circuit

```
circom --r1cs --wasm --c --sym --inspect \
  --prime bls12381 circuit.circom -o .
```

Run this command from the directory where your `include` path can reach the `circmlib` node module. For Janus Wallet this is `src/zkproof/janus-wallet/`.

The flags produce:

- `circuit.r1cs`: the constraint system in binary format.
- `circuit_js/circuit.wasm`: WebAssembly witness calculator.
- `circuit.sym`: symbol table for debugging.

The `--prime bls12381` flag targets the BLS12-381 curve, which matches Cardano's on-chain primitives.

Inspect the compiled circuit:

```
snarkjs r1cs info circuit.r1cs
```

For the Janus circuit you should see:

```
[INFO] snarkJS: Curve: bls12-381
[INFO] snarkJS: # of Wires: 1357
[INFO] snarkJS: # of Constraints: 1352
[INFO] snarkJS: # of Private Inputs: 1
[INFO] snarkJS: # of Public Inputs: 5
[INFO] snarkJS: # of Outputs: 0
```

5.3. THE TRUSTED SETUP CEREMONY

Groth16 requires a two-phase trusted setup. Phase 1 (Powers of Tau) is circuit-independent and can be reused across projects. Phase 2 is circuit-specific and ties the parameters to your exact R1CS.

Phase 1: Powers of Tau

1. START A NEW CEREMONY.

```
snarkjs powersoftau new bls12381 14 pot14_0000.ptau -v
```

The second argument (14) is the power of two of the maximum number of constraints: $2^{14} = 16,384$. Choose the smallest power that fits your circuit to keep file sizes manageable. The maximum supported is $2^{28} \approx 268$ million constraints.

2-3. CONTRIBUTE RANDOMNESS.

Multiple contributors make the ceremony secure: the parameters are safe as long as *at least one* contributor honestly discarded their secret after contributing.

```
snarkjs powersoftau contribute pot14_0000.ptau pot14_0001.ptau \
  --name="First contribution" -v
```

```
snarkjs powersoftau contribute pot14_0001.ptau pot14_0002.ptau \
  --name="Second contribution" -v -e="some random text"
```

The `-e` flag provides entropy inline, making the step non-interactive, useful in scripts.

4. (OPTIONAL) THIRD-PARTY CONTRIBUTION.

You can mix contributions from different software:

```
snarkjs powersoftau export challenge pot14_0002.ptau challenge_0003
```

```
snarkjs powersoftau challenge contribute bls12381 challenge_0003 \
  response_0003 -e="some random text"
```

```
snarkjs powersoftau import response pot14_0002.ptau response_0003 \
  pot14_0003.ptau -n="Third contribution name"
```

5. VERIFY THE CEREMONY SO FAR.

```
snarkjs powersoftau verify pot14_0003.ptau
```

A clean run ends with `Powers Of tau OK!`

6. APPLY A RANDOM BEACON.

A random beacon finalises Phase 1. It is a delayed hash function evaluated on a publicly known, high-entropy value: for example, the hash of a specific Cardano block, or a national lottery result announced after the ceremony began.

```
snarkjs powersoftau beacon pot14_0003.ptau pot14_beacon.ptau \
  3c692631872308ff1f9fed102b940aecee160def9c2478b50b977091 \
  10 -n="Final Beacon"
```

The 10 argument means $2^{10} = 1024$ iterations of the hash, enough to make the beacon computationally irreversible.

Why a random beacon? If a contributor could predict the beacon before their contribution, they might be able to bias the parameters. An unpredictable, publicly verifiable source of randomness prevents this. Janus Wallet uses the `policy_id` under which all account tokens are minted as its beacon value.

7. PREPARE PHASE 2.

```
snarkjs powersoftau prepare phase2 pot14_beacon.ptau pot14_final.ptau -v
```

This pre-computes the encrypted Lagrange polynomial evaluations needed in Phase 2.

8. FINAL VERIFICATION.

```
snarkjs powersoftau verify pot14_final.ptau
```

Unlike step 5, this run will *not* warn about missing phase 2 values; that means the file is ready.

Phase 2: Circuit-Specific Setup

9-10. COMPILE THE CIRCUIT AND GENERATE T

```
snarkjs groth16 setup circuit.r1cs pot14_final.ptau circuit_0000.zkey
```

Do not use `circuit_0000.zkey` in production. It has zero contributions and is not safe.

11-12. ADD CONTRIBUTIONS TO PHASE 2.

```
snarkjs zkey contribute circuit_0000.zkey circuit_0001.zkey \
  --name="1st Contributor Name" -v

snarkjs zkey contribute circuit_0001.zkey circuit_0002.zkey \
  --name="Second contribution" -v -e="Another random entropy"
```

13. (OPTIONAL) THIRD-PARTY PHASE 2 CONTR

```
snarkjs zkey export bellman circuit_0002.zkey challenge_phase2_0003
snarkjs zkey bellman contribute bls12381 challenge_phase2_0003 \
  response_phase2_0003 -e="some random text"
snarkjs zkey import bellman circuit_0002.zkey response_phase2_0003 \
  circuit_0003.zkey -n="Third contribution name"
```

14. VERIFY THE ZKEY.

```
snarkjs zkey verify circuit.r1cs pot14_final.ptau circuit_0003.zkey
```

Output: [INFO] snarkJS: ZKey Ok!

15. APPLY THE RANDOM BEACON AND EXPORT T

```
snarkjs zkey beacon circuit_0003.zkey circuit_final.zkey \
  0102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f \
  10 -n="Final Beacon phase2"
```

```
snarkjs zkey verify circuit.r1cs pot14_final.ptau circuit_final.zkey
```

```
snarkjs zkey export verificationkey circuit_final.zkey verification_key.json
```

verification_key.json contains the public parameters that will be embedded in the Aiken smart contract (as an on-chain token) and used to verify every proof.

5.4. GENERATING AND VERIFYING A PROOF

Prepare Inputs

Create an input.json with the circuit's signals. Note that large integers must be quoted strings to avoid JSON precision loss:

```
{
  "userId": "52435875175126190479447740508185965837690552500527637822...",
  "credentialHash": "17469782945530594987388377835453341901430155623074...",
  "challenge": "17469782945530594987388377835453341901430155623074104...",
  "challengeFlag": "1",
  "circuitHash": "24883872159189864133794802823127472657146334491119345...",
  "pwd": "52435875175126190479447740508185965837690552500527637822..."
}
```

Calculate the Witness

The witness is the complete set of wire values for all signals in the circuit given your specific inputs.

```
snarkjs wtms calculate circuit_js/circuit.wasm input.json witness.wtns
snarkjs wtms check circuit.r1cs witness.wtns
```

A valid witness prints WITNESS IS CORRECT.

Generate the Proof

```
snarkjs groth16 prove circuit_final.zkey witness.wtns proof.json public.json
```

This produces:

- `proof.json`: the Groth16 proof (π_A , π_B , π_C elliptic curve points).
- `public.json`: the values of the public inputs (visible to the verifier).

Or in a single step:

```
snarkjs groth16 fullprove input.json circuit_js/circuit.wasm \
  circuit_final.zkey proof.json public.json
```

Verify Off-Chain

```
snarkjs groth16 verify verification_key.json public.json proof.json
```

Output: [INFO] snarkJS: OK!

5.5. PREPARING THE VERIFICATION KEY FOR CARDANO

Cardano's BLS12-381 primitives work exclusively with **compressed** elliptic curve points. The `verification_key.json` produced by `snarkjs` uses *uncompressed* points. Before moving on-chain you must compress all $\mathbb{G}_1$ and $\mathbb{G}_2$ points: `vk_alpha_1`, `vk_beta_2`, `vk_gamma_2`, `vk_delta_2`, and every point in the IC array.

Janus Wallet provides a script for this:

```
node --no-warnings --experimental-specifier-resolution=node \
  --loader ts-node/esm compress_zk_verification_key.ts \
  janus-wallet/verification_key.json \
  janus-wallet/compressed_verification_key.json
```

Run this from `src/zkproof`. The output `compressed_verification_key.json` is what goes on-chain.

5.6. ON-CHAIN VERIFICATION IN AIKEN

The verification key is minted as a token on Cardano using Janus Wallet's circuit minting endpoint. This makes the parameters immutable and publicly auditable; anyone can inspect the token and verify the ceremony.

Minting the Circuit Token

```
POST http://localhost:3001/circuits
{
  "token_name": "Circuit#000",
  "version": 0,
  "nonce": "some random value",
  "circuit": {
    "vk_alpha1": "8f944ea2...",
    "vk_beta2": "81dfe069...",
    "vk_gamma2": "93e02b60...",
    "vk_delta2": "910d1b3d...",
    "vk_ic": [
      "8d043ff5...",
      "86fa74a9...",
      ...
    ]
  }
}
```

The endpoint mints the token under a `policy_id` derived from the nonce. The nonce can be the beacon value used in Phase 2, linking the on-chain token cryptographically to the ceremony transcript.

How the Aiken Validator Works

The Aiken validator performs a Groth16 pairing check using the BLS12-381 primitives:

1. It reads the compressed verification key from the circuit token in the reference input.
2. It reads the proof (π_A, π_B, π_C) from the redeemer.

3. It reads the public inputs (`userId`, `credentialHash`, `challenge`, `challengeFlag`, `circuitHash`) from the datum.
4. It computes the pairing equation and calls `final_verify` to check the proof.
5. If the proof is valid, the UTxO is unlocked. If not, the transaction fails.

The Aiken test suite from Janus Wallet shows all passing scenarios:

```
aiken check -t silent .
```

```
+-- tests/verify -----
| PASS [mem: 89791, cpu: 2541340679] verify_zk_proof
| PASS [mem: 89791, cpu: 2541340679] verify_zk_proof_1
| PASS [mem: 63964, cpu: 1355803465] verify_zk_proof_invalid_user
| PASS [mem: 63964, cpu: 1355812874] verify_zk_proof_invalid_circuit_hash
| PASS [mem: 63964, cpu: 1355812874] verify_zk_proof_invalid_challenge
| PASS [mem: 63964, cpu: 1355812874] verify_zk_proof_invalid_proof
+----- 11 tests | 11 passed | 0 failed
```

Notice the CPU budget for `verify_zk_proof`: approximately 2.5 billion units. Groth16 verification on BLS12-381 is one of the most expensive operations you can perform on-chain, but it is within Cardano's per-transaction budget, which is why Plutus V3 added these primitives in the first place.

5.7. JANUS WALLET: PUTTING IT ALL TOGETHER

Janus Wallet (<https://github.com/leobel/janus-wallet>) is the reference implementation of a ZK smart contract wallet on Cardano. Instead of signing transactions with a private key, the user proves knowledge of their password via a Groth16 circuit. The private key never exists; there is nothing to lose or steal.

Architecture

- **Circuit**: the Circom Authenticate template described above.
- **Off-chain server**: a Node.js HTTP API that builds, signs (via ZK proof), and submits transactions.
- **On-chain validator**: an Aiken smart contract that verifies the Groth16 proof against the circuit token.
- **PostgreSQL**: stores user accounts, circuit data, and UTxO state.

Running the Server

Create a `.env` file:

```
PORT=3001
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=janus
```



```

DB_PASSWORD=123456
DB_NAME=janus_wallet
CARDANO_NETWORK=Preview
BLOCKFROST_API_URL=https://cardano-preview.blockfrost.io/api/v0
BLOCKFROST_API_KEY=preview...
WALLET_PRV_KEY=ed25519_sk...
COLLATERAL_PRV_KEY=ed25519_sk...
COLLATERAL_ADDRESS=addr_test...

```

Run database migrations and start the server:

```

npm run migrate
npm start

```

The signing logic that produces the ZK proof depends on `circom` being accessible from the process running the server. Verify with `circom --version`. If the binary is at `$HOME/.cargo/bin` and not on your `PATH`, copy it to `/usr/local/bin`.

User Lifecycle

REGISTER A USER.

```

POST http://localhost:3001/wallets/{user_name}
{
  "hash":      "42b79954...", // Poseidon(password, username)
  "kdf_hash": "243262..."   // bcrypt(password, salt)
}

```

The server generates a smart contract address unique to this user and mints a profile NFT to it. The `hash` is the on-chain credential; the `kdf_hash` lets the server authenticate requests without storing the plaintext password.

SEND FUNDS (THREE STEPS).

```

// 1. Build the unsigned transaction
POST http://localhost:3001/wallets/{user_id}/build
{ "amount": 5000000, "receive_address": "addr_test1..." }
// Response: { "tx": "84a900..." }

// 2. Sign with the ZK proof
POST http://localhost:3001/wallets/{user_id}/sign
{ "pwd": "supersecret", "tx": "84a900..." }

```

```
// 3. Submit
POST http://localhost:3001/wallets/{user_id}/send
{
  "redeemers": ["d8799f..."], // ZK proof as CBOR
  "tx": "84a900..."
}
```

The `redeemers` array contains the Groth16 proof serialised as CBOR. Additional redeemers in the array are bypass markers for UTxOs from the same wallet that do not require a fresh proof.

STAKE, DELEGATE, AND WITHDRAW.

Janus Wallet supports the full Cardano staking lifecycle through the same three-step pattern (build → sign → send):

```
POST /wallets/{user_id}/pools/{pool_id}/registerAndDelegate
POST /wallets/{user_id}/pools/{pool_id}/delegate
POST /wallets/{user_id}/dreps/{drep_id}/delegate
POST /wallets/{user_id}/withdraw { "amount": 1234567 }
```

The key insight: from the blockchain's perspective, every Janus transaction is a normal smart contract spend. The only thing that makes it a ZK wallet is the redeemer, a Groth16 proof that the user knows their password, checked on-chain by the Aiken validator in a single pairing verification.

Full Test Results

Janus Wallet ships with 29 Aiken tests covering minting, spending, challenge construction, and ZK verification; all passing with zero errors:

```
Summary 29 checks, 0 errors, 19 warnings
```

The warnings are informational (unused variables in test helpers) and do not affect correctness.

This is the complete picture of ZK on Cardano today: Circom defines the computation, snarkjs runs the ceremony and generates proofs, and Aiken verifies them on-chain using native BLS12-381 pairing operations. In the next chapters we will build our own circuit and validator from scratch.

6. PASSWORD-LOCKED UTXO WITH GROTH16

6.1. WHAT WE ARE BUILDING

This chapter builds the simplest possible ZK smart contract on Cardano: a UTxO that can only be unlocked by someone who can *prove* they know a secret password, without ever revealing the password on-chain.

The password is hashed with **Poseidon**, a ZK-friendly hash function. The hash is stored publicly in the datum. To spend the UTxO, the spender generates a Groth16 proof that they know a pre-image of that hash. The Aiken validator verifies the proof on-chain using the BLS12-381 pairing primitives introduced in Plutus V3.

What stays private: the password. **What goes on-chain:** the Poseidon hash, the verification key, and the proof.

The complete source lives in `zk-password/` at the root of the repository. By the end of this chapter the directory will look like this:

```
zk-password/
+-- aiken.toml
+-- package.json
+-- circuit/
|   +-- circuit.circom      <- Circom constraint circuit
|   +-- poseidon_hash.circom <- helper: compute hash without constraint
|   +-- gen_input.js        <- script: password -> input.json
|   +-- compress_vk.js      <- script: compress verification key
|   +-- compress_proof.js   <- script: compress proof points
+-- lib/
|   +-- zk_password/
|       +-- groth16.ak      <- Groth16 verifier library
|       +-- password_tests.ak <- Aiken unit tests
+-- validators/
|   +-- password.ak        <- on-chain validator
```

6.2. PROJECT SETUP

Create the Aiken project and initialise npm for the JavaScript utility scripts:

```
aiken new chapter6/zk-password
cd zk-password
npm init -y
npm install -D circomlib
npm install @noble/curves
```

`circomlib` provides the Poseidon circuit templates. `@noble/curves` is used by the compression scripts to convert uncompressed BLS12-381 points to the 48-byte (G1) and

96-byte (G2) format that Cardano accepts.

6.3. THE CIRCUIT

Create `circuit/circuit.circom`:

```
pragma circom 2.1.6;

include "../node_modules/circomlib/circuits/poseidon.circom";

// Prove knowledge of a password whose Poseidon hash matches a commitment.
//
// Public inputs (visible on-chain in the datum):
//   hash -- Poseidon(pwd)
//
// Private inputs (never leave the prover's device):
//   pwd  -- the password
template PasswordLock() {
    signal input hash;    // public
    signal input pwd;     // private

    component h = Poseidon(1);
    h.inputs[0] <== pwd;

    hash === h.out;
}

component main {public [hash]} = PasswordLock();
```

The `===` operator defines an arithmetic constraint: if the Poseidon hash of `pwd` does not equal `hash`, the constraint system is unsatisfiable and no valid proof can ever be constructed. The `<==` operator assigns a value and simultaneously constrains the signal.

Compile the Circuit

From inside `zk-password/`:

```
circom --r1cs --wasm --sym --inspect \
    --prime bls12381 circuit/circuit.circom -o circuit/
```

The `--prime bls12381` flag targets the BLS12-381 scalar field, which matches Cardano's on-chain pairing primitives. The outputs land in `circuit/`:

- `circuit.r1cs`: the binary constraint system.
- `circuit_js/circuit.wasm`: the WebAssembly witness calculator.
- `circuit.sym`: the symbol table (signal names and witness indices).

Inspect the compiled R1CS:

```

template instances: 71
non-linear constraints: 216
linear constraints: 199
public inputs: 1
private inputs: 1
public outputs: 0
wires: 417
labels: 583
Written successfully: circuit/circuit.r1cs
Written successfully: circuit/circuit.sym
Written successfully: circuit/circuit_js\circuit.wasm
Everything went okay

```

Circom compilation output. Our circuit has 216 non-linear constraints, 1 public input (hash), and 1 private input (pwd).

```
snarkjs r1cs info circuit/circuit.r1cs
```

You should see Curve: bls12-381, 1 public input, 1 private input, and roughly 240 constraints (mostly Poseidon round internals).

6.4. GENERATING INPUT.JSON

The circuit needs two inputs: the password as a field element and its Poseidon hash. There is a subtlety here worth understanding before writing any code.

The Field-Element Problem

The pwd signal in the circuit is a BLS12-381 field element, an integer, not a string. A plain numeric string like "12345" converts trivially with `BigInt("12345")`. A word like "mypassword" does not, and calling `BigInt("mypassword")` throws a runtime error.

The solution is to encode the string as a big-endian integer of its UTF-8 bytes:

```

function toFieldElement(s) {
  if (/^\d+$/.test(s)) return BigInt(s);
  return BigInt("0x" + Buffer.from(s, "utf8").toString("hex"));
}

```

The Hash Computation Problem

We also need the Poseidon hash of the password, but we cannot just call `circomlibjs`'s `buildPoseidon` here. That library implements Poseidon over the **BN128** field; our circuit uses the **BLS12-381** scalar field. The two fields have different primes, so the hash values differ.

We solve this by compiling a tiny helper circuit that simply exposes the hash as an output signal; no equality constraint, just a wire:

```
circuit/poseidon_hash.circom:
```

```
pragma circom 2.1.6;

include "../node_modules/circomlib/circuits/poseidon.circom";

template HashOnly() {
    signal input  pwd;
    signal output hash;
    component h = Poseidon(1);
    h.inputs[0] <== pwd;
    hash <== h.out;
}

component main = HashOnly();
```

Compile it with the same BLS12-381 prime:

```
circom --wasm --prime bls12381 \
    circuit/poseidon_hash.circom -o circuit/
```

Because this circuit has no `===` constraint, the witness calculator runs cleanly for any input and `witness[1]`, the first public output, contains `Poseidon(pwd)` computed over the correct field.

circuit/gen_input.js

```
const { readFileSync, writeFileSync } = require("fs");
const path = require("path");

const password = process.argv[2] ?? "12345";

function toFieldElement(s) {
    if (/^\d+$/.test(s)) return BigInt(s);
    return BigInt("0x" + Buffer.from(s, "utf8").toString("hex"));
}

const pwd = toFieldElement(password);

async function main() {
    const buildWC = require("../poseidon_hash_js/witness_calculator.js");
    const wasm = readFileSync(
        path.join(__dirname, "poseidon_hash_js", "poseidon_hash.wasm")
    );
    const wc = await buildWC(wasm);

    const witness = await wc.calculateWitness({ pwd }, true);
    const hash = witness[1]; // output signal is always at index 1

    const input = { hash: hash.toString(), pwd: pwd.toString() };
    const outPath = path.join(__dirname, "input.json");
    writeFileSync(outPath, JSON.stringify(input, null, 2));
}
```

```

    console.log("password :", password);
    console.log("hash      :", hash.toString());
    console.log("written   :", outPath);
  }

  main().catch(err => { console.error(err); process.exit(1); });

```

Usage:

```

node circuit/gen_input.js 123456
node circuit/gen_input.js mypassword
node circuit/gen_input.js "hello world"

```

The resulting `circuit/input.json` contains the hash and password as quoted decimal strings, ready for `snarkjs groth16 fullprove`.

6.5. THE TRUSTED SETUP

Groth16 requires a two-phase trusted setup. Phase 1 (Powers of Tau) is circuit-independent; Phase 2 is specific to your R1CS.

Phase 1: Powers of Tau

```

snarkjs powersoftau new bls12381 12 pot12_0000.ptau -v
snarkjs powersoftau contribute pot12_0000.ptau pot12_0001.ptau \
  --name="dev" -e="some entropy"
snarkjs powersoftau beacon pot12_0001.ptau pot12_beacon.ptau \
  00000000000000000000000000000000 10 -n="Dev beacon"
snarkjs powersoftau prepare phase2 pot12_beacon.ptau pot12_final.ptau -v

```

We use $2^{12} = 4096$ max constraints, well above our 216-constraint circuit and fast to generate. Use a larger power (e.g. 14) for bigger circuits.

Phase 2: Circuit-Specific Setup

```

snarkjs groth16 setup circuit/circuit.r1cs pot12_final.ptau zkey_0000.zkey
snarkjs zkey contribute zkey_0000.zkey zkey_0001.zkey \
  --name="dev" -e="more entropy"
snarkjs zkey beacon zkey_0001.zkey zkey_final.zkey \
  00000000000000000000000000000000 10 -n="Final beacon"
snarkjs zkey export verificationkey zkey_final.zkey verification_key.json

```

For production you should use an existing BLS12-381 Powers of Tau file (e.g. from the Zcash ceremony) and run at least three independent Phase 2 contributions. The parameters here are safe only for local development.


```

[INFO] snarkJS: Circuit Hash:
        53b173e7 5f0fa4a9 cde8c117 4812a0a0
        7ff9248a 2763bba1 03a792f5 8eac5f2f
        84d032d4 fdad9a37 0c32f55b ac123c23
        bf0b1461 b1ac49ec b0b9d223 1dec1dfe
[INFO] snarkJS: Contribution Hash:
        b648e692 31a53b0c 0322b310 c4619b65
        c6bce647 31d34b08 ba492013 c75de9d2
        c2602285 1684b22c d7d37f6d 82f5cb68
        bf57d0c7 60c11f28 85343355 0c5673fc
PS C:\Users\raulr\Documents\GitHub\ZK-from-zero-on-Cardano\zk-password> snarkjs groth16 fullprove \
-n="Final beacon"
[INFO] snarkJS: Contribution Hash:
        0fed237b ed8f4342 064d246a d5add71b
        f4de0314 61fba467 14aafec8 fa826e31
        fd46cf7b e2a98810 cb615e83 56561d83
        55c7f100 cbeff94d f2c7db3b f37ff1a4
PS C:\Users\raulr\Documents\GitHub\ZK-from-zero-on-Cardano\zk-password> snarkjs groth16 exportVerificationKey \
[INFO] snarkJS: EXPORT VERIFICATION KEY STARTED
[INFO] snarkJS: > Detected protocol: groth16
[INFO] snarkJS: EXPORT VERIFICATION KEY FINISHED

```

snarkjs exports the verification key to `verification_key.json`. This file contains the uncompressed BLS12-381 G1 and G2 points.

6.6. GENERATING THE PROOF

With `circuit/input.json`, the compiled WASM, and the final zkey, generate the proof in one command:

```
snarkjs groth16 fullprove \
  circuit/input.json \
  circuit/circuit_js/circuit.wasm \
  zkey_final.zkey \
  proof.json public.json
```

Verify the proof off-chain before going further:

```
snarkjs groth16 verify verification_key.json public.json proof.json
```

You should see `[INFO] snarkJS: OK!`. The `public.json` file contains the single public input: the Poseidon hash of your password.

6.7. COMPRESSING POINTS FOR CARDANO

Cardano's BLS12-381 builtins only accept **compressed** elliptic-curve points. The snarkjs output uses uncompressed Weierstrass coordinates $[x, y, 1]$ for G1 and $[[x_0, x_1], [y_0, y_1], [1, 0]]$ for G2. Compressed form encodes a point in its x -coordinate alone (48 bytes for G1, 96 bytes for G2), using one flag bit to disambiguate which of the two possible y values to use.

We compress both the verification key and the proof with the `@noble/curves` library.

```
circuit/compress_vk.js
```

```
const { readFileSync, writeFileSync } = require("fs");
const path = require("path");

const vkPath = process.argv[2]
  ?? path.join(__dirname, "..", "verification_key.json");
const outPath = process.argv[3]
  ?? path.join(__dirname, "..", "compressed_vk.json");

async function main() {
  const { bls12_381 } = await import("@noble/curves/bls12-381.js");
  const G1 = bls12_381.G1.Point;
  const G2 = bls12_381.G2.Point;

  const vk = JSON.parse(readFileSync(vkPath, "utf8"));

  function compressG1([x, y]) {
    return Buffer.from(
      G1.fromAffine({ x: BigInt(x), y: BigInt(y) }).toBytes(true)
    ).toString("hex");
  }

  function compressG2([[x0, x1], [y0, y1]]) {
    return Buffer.from(
      G2.fromAffine({
        x: { c0: BigInt(x0), c1: BigInt(x1) },
        y: { c0: BigInt(y0), c1: BigInt(y1) },
      }).toBytes(true)
    ).toString("hex");
  }

  const compressed = {
    vk_alpha1: compressG1(vk.vk_alpha_1),
    vk_beta2: compressG2(vk.vk_beta_2),
    vk_gamma2: compressG2(vk.vk_gamma_2),
    vk_delta2: compressG2(vk.vk_delta_2),
    vk_ic: vk.IC.map(compressG1),
  };

  writeFileSync(outPath, JSON.stringify(compressed, null, 2));
}

main().catch(err => { console.error(err); process.exit(1); });
```

node circuit/compress_vk.js

The resulting `compressed_vk.json` has:

- `vk_alpha1`: 96 hex chars (48 bytes, compressed G1)
- `vk_beta2`, `vk_gamma2`, `vk_delta2`: 192 hex chars each (96 bytes, compressed G2)
- `vk_ic`: array of two 96-hex-char strings; the first element is the constant term IC_0 , the second is IC_1 (one entry per public input)

circuit/compress_proof.js

The proof points use the same approach:

```
const { readFileSync, writeFileSync } = require("fs");
const path = require("path");

const proofPath = process.argv[2]
  ?? path.join(__dirname, "..", "proof.json");
const outPath = process.argv[3]
  ?? path.join(__dirname, "..", "compressed_proof.json");

async function main() {
  const { bls12_381 } = await import("@noble/curves/bls12-381.js");
  const G1 = bls12_381.G1.Point;
  const G2 = bls12_381.G2.Point;

  const proof = JSON.parse(readFileSync(proofPath, "utf8"));

  const compressed = {
    pi_a: Buffer.from(
      G1.fromAffine({
        x: BigInt(proof.pi_a[0]),
        y: BigInt(proof.pi_a[1]),
      }).toBytes(true)
    ).toString("hex"),
    pi_b: Buffer.from(
      G2.fromAffine({
        x: { c0: BigInt(proof.pi_b[0][0]), c1: BigInt(proof.pi_b[0][1]) },
        y: { c0: BigInt(proof.pi_b[1][0]), c1: BigInt(proof.pi_b[1][1]) },
      }).toBytes(true)
    ).toString("hex"),
    pi_c: Buffer.from(
      G1.fromAffine({
        x: BigInt(proof.pi_c[0]),
        y: BigInt(proof.pi_c[1]),
      }).toBytes(true)
    ).toString("hex"),
  };

  writeFileSync(outPath, JSON.stringify(compressed, null, 2));
}

main().catch(err => { console.error(err); process.exit(1); });
```

node circuit/compress_proof.js

6.8. THE GROTH16 VERIFIER LIBRARY

The file `lib/zk_password/groth16.ak` is a self-contained Groth16 verifier modelled after Janus Wallet's implementation.

Types

```
pub type VerificationKey {
  alpha: ByteArray,      // G1: 48 bytes
  beta:  ByteArray,      // G2: 96 bytes
  gamma: ByteArray,      // G2: 96 bytes
  delta: ByteArray,      // G2: 96 bytes
  ic:    List<ByteArray>, // G1 points; length = num_public_inputs + 1
}

pub type Proof {
  pi_a: ByteArray, // G1: 48 bytes
  pi_b: ByteArray, // G2: 96 bytes
  pi_c: ByteArray, // G1: 48 bytes
}
```

All fields are compressed bytes. The `ic` list has two elements for a circuit with one public input: the constant term IC_0 and the coefficient IC_1 for hash.

The Verification Function

The pairing equation that Groth16 checks is:

$$e(\pi_A, \pi_B) = e(\alpha, \beta) \cdot e(\mathbf{vk_x}, \gamma) \cdot e(\pi_C, \delta)$$

where $\mathbf{vk_x} = IC_0 + \sum_i (\text{public_input}_i \cdot IC_{i+1})$.

```
pub fn groth16_verify(
  vk: VerificationKey,
  proof: Proof,
  public_inputs: List<Scalar>,
) -> Bool {
  let a = g1.decompress(proof.pi_a)
  let b = g2.decompress(proof.pi_b)
  let c = g1.decompress(proof.pi_c)

  let vk_x =
    when vk.ic is {
      [] -> fail @"vk.ic must not be empty"
      [ic0, ..rest] -> derive(rest, public_inputs, g1.decompress(ic0))
    }

  let e_ab          = bls12_381_miller_loop(a, b)
  let e_alpha_beta =
    bls12_381_miller_loop(g1.decompress(vk.alpha), g2.decompress(vk.beta))
  let e_x_gamma     = bls12_381_miller_loop(vk_x, g2.decompress(vk.gamma))
  let e_c_delta     = bls12_381_miller_loop(c, g2.decompress(vk.delta))
```

```

let rhs =
  bls12_381_mul_miller_loop_result(
    bls12_381_mul_miller_loop_result(e_alpha_beta, e_x_gamma),
    e_c_delta,
  )

  bls12_381_final_verify(e_ab, rhs)
}

```

The entire cryptographic work happens inside Cardano's builtin functions (`bls12_381_miller_loop` and `bls12_381_final_verify`). These are native operations in Plutus V3 and are priced at fixed, low execution units regardless of proof complexity.

6.9. THE AIKEN VALIDATOR

The Opaque Scalar Problem

A natural first attempt for the datum would be:

```

pub type Datum {
  credential_hash: Scalar, -- would be natural, but WRONG
  vk: VerificationKey,
}

```

The Aiken compiler immediately rejects this:

```

x I caught an opaque type trying to escape
|
| datum: Option<Datum>,
| ----- invalid validator's boundary
|
| = Opaque types cannot figure anywhere in an outward-facing type
|   like a validator's redeemer or datum.

```

`Scalar` is *opaque*: its constructor is hidden and it enforces invariants that cannot be expressed as plain Plutus Data. Opaque types cannot cross the on-chain/off-chain boundary (datum or redeemer), because the off-chain world has no way to build them safely from raw bytes.

The fix: store the hash as a plain `Int` in the datum (a normal Plutus integer that any off-chain tool can construct), and convert it to a `Scalar` at runtime inside the validator body using `scalar.new`:

```

use aiken/crypto/bls12_381/scalar
use cardano/transaction.{OutputReference, Transaction}
use zk_password/groth16.{Proof, VerificationKey, groth16_verify}

```

```

pub type Datum {
    credential_hash: Int,    // Poseidon(pwd) as a plain integer
    vk: VerificationKey,    // compressed Groth16 verification key
}

pub type Redeemer {
    proof: Proof,           // compressed Groth16 proof
}

validator password_lock {
    spend(
        datum: Option<Datum>,
        redeemer: Redeemer,
        _self: OutputReference,
        _ctx: Transaction,
    ) {
        expect Some(d) = datum
        // Convert Int -> Scalar at runtime. scalar.new returns None if
        // the value is outside [0, field_prime), which rejects malformed datums.
        expect Some(hash_scalar) = scalar.new(d.credential_hash)
        groth16_verify(d.vk, redeemer.proof, [hash_scalar])
    }

    else(_) {
        False
    }
}

```

The datum stores two things: the Poseidon hash of the password (the lock) and the verification key (the lock's mechanism). Both are public. The redeemer provides the proof (the key). The key never reveals the password.

Compile

```
aiken check
```

You should see zero errors and zero warnings.

6.10. TESTING WITH AIKEN

Create `lib/zk_password/password_tests.ak` with two tests: one that asserts a valid proof passes, and one that asserts a wrong hash fails.

```

use aiken/crypto/bls12_381/scalar
use zk_password/groth16.{Proof, VerificationKey, groth16_verify}

fn test_vk() -> VerificationKey {
    VerificationKey {

```

```

        alpha: #"<vk_alpha1 from compressed_vk.json>",
        beta:  #"<vk_beta2  from compressed_vk.json>",
        gamma: #"<vk_gamma2 from compressed_vk.json>",
        delta: #"<vk_delta2 from compressed_vk.json>",
        ic: [
            #"<vk_ic[0] from compressed_vk.json>",
            #"<vk_ic[1] from compressed_vk.json>",
        ],
    }
}

fn test_proof() -> Proof {
    Proof {
        pi_a: #"<pi_a from compressed_proof.json>",
        pi_b: #"<pi_b from compressed_proof.json>",
        pi_c: #"<pi_c from compressed_proof.json>",
    }
}

// Public signal from public.json (the hash snarkjs used to generate the proof)
const credential_hash: Int =
    9415434746508747181228645764267412889835803815708024285868314342086739058356

test groth16_verify_valid_proof() {
    expect Some(hash_scalar) = scalar.new(credential_hash)
    groth16_verify(test_vk(), test_proof(), [hash_scalar])
}

test groth16_verify_wrong_hash() fail {
    let wrong_hash = credential_hash + 1
    expect Some(hash_scalar) = scalar.new(wrong_hash)
    groth16_verify(test_vk(), test_proof(), [hash_scalar])
}

```

Fill in the placeholder hex strings with the values from your `compressed_vk.json` and `compressed_proof.json`. The `credential_hash` is the integer in `public.json`.

aiken check

Expected output:

```

Testing ...
PASS  zk_password/password_tests :: groth16_verify_valid_proof
PASS  zk_password/password_tests :: groth16_verify_wrong_hash
Summary: 2 passed, 0 failed

```

The `groth16_verify_wrong_hash` test is marked **fail**, meaning it is expected to fail. Aiken marks it as passing precisely *because* the verification rejects the wrong hash, confirming the validator cannot be fooled.

6.11. OFF-CHAIN CODE WITH MESHJS

With the validator compiled and tested, we need off-chain code to build the lock and unlock transactions. We use **MeshJS**, a TypeScript SDK for Cardano.

```
npm install @meshsdk/core
```

Read the compiled blueprint and the compressed keys:

```
import fs from "fs";
import {
  MeshWallet,
  Transaction,
  resolveScriptHash,
  serializePlutusScript,
  applyParamsToScript,
} from "@meshsdk/core";
import blueprint from "./plutus.json" assert { type: "json" };
import compressedVk from "./compressed_vk.json" assert { type: "json" };
import compressedProof from "./compressed_proof.json" assert { type: "json" };

// Helper: hex string -> Mesh ByteString value
const hex = (s) => ({ alternative: 0, fields: [s] });

// VerificationKey as Plutus Data (mirrors the Aiken record field order)
const vkData = {
  alternative: 0,
  fields: [
    compressedVk.vk_alpha1,
    compressedVk.vk_beta2,
    compressedVk.vk_gamma2,
    compressedVk.vk_delta2,
    compressedVk.vk_ic, // List<ByteArray>
  ],
};
```

Locking a UTx0

To lock funds, build a transaction that sends ADA to the script address with the datum encoding (credential_hash, vk):

```
import { publicSignals } from "./public.json" assert { type: "json" };

const credentialHash = publicSignals[0]; // decimal string from public.json

// Datum: Constr(0, [Int(credentialHash), VerificationKey])
const datum = {
  alternative: 0,
  fields: [
```



```

        parseInt(credentialHash), // credential_hash as Int
        vkData,                    // vk: VerificationKey
    ],
};

const script = {
    code: blueprint.validators[0].compiledCode,
    version: "V3",
};

const scriptAddress = serializePlutusScript(script, undefined, 0);

const wallet = new MeshWallet({
    networkId: 0, // 0 = testnet, 1 = mainnet
    fetcher: provider,
    submitter: provider,
    key: { type: "mnemonic", words: yourMnemonic },
});

const tx = new Transaction({ initiator: wallet })
    .sendLovelace(
        { address: scriptAddress, datum: { value: datum, inline: true } },
        "5000000" // 5 ADA
    );

const signed = await wallet.signTx(await tx.build());
const txHash = await wallet.submitTx(signed);
console.log("Locked:", txHash);

```

The datum is stored **inline** on-chain. Anyone can read the Poseidon hash and the verification key from the UTxO. Neither reveals the password.

Unlocking a UTxO

To unlock, the spender must have generated a valid proof off-chain (using `gen_input.js` + `snarkjs groth16 fullprove + compress_proof.js`). They then build a transaction with the redeemer encoding the compressed proof:

```

// Redeemer: Constr(0, [Proof])
const redeemer = {
    data: {
        alternative: 0,
        fields: [{
            alternative: 0,
            fields: [
                compressedProof.pi_a,
                compressedProof.pi_b,
                compressedProof.pi_c,
            ],
        }],
    },
};

```

```

    },
    budget: { mem: 14000000, cpu: 10000000000 },
  };

  // Fetch the locked UTxO at the script address
  const utxos = await provider.fetchAddressUTxOs(scriptAddress);
  const lockedUtxo = utxos[0];

  const tx = new Transaction({ initiator: wallet })
    .redeemValue({
      value: lockedUtxo,
      script,
      redeemer,
    })
    .sendLovelace(await wallet.getChangeAddress(), "4800000");

  const signed = await wallet.signTx(await tx.build(), true);
  const txHash = await wallet.submitTx(signed);
  console.log("Unlocked:", txHash);

```

When this transaction is submitted, the Cardano node evaluates the `password_lock` validator. The validator decompresses the proof and VK, calls `groth16_verify`, and approves the spend only if the pairing equation holds, which it will if and only if the spender knows the correct password.

The password is used only *locally* to run `fullprove`. It never appears in the transaction, the redeemer, or anywhere on-chain. An observer who inspects the blockchain sees the compressed proof and the hash, neither reveals the password.

6.12. END-TO-END SUMMARY

Here is the complete workflow from password to unlocked UTxO:

1. **Setup** (once per circuit): compile `circuit.circom`, run the trusted setup ceremony, export `verification_key.json`, compress it to `compressed_vk.json` with `compress_vk.js`.
2. **Lock**: choose a password, run `node circuit/gen_input.js <password>` to produce `circuit/input.json`, then lock a UTxO with the datum (`credential_hash`, `vk`).
3. **Prove**: run `snarkjs groth16 fullprove` with the `input.json`, the WASM, and `zkey_final.zkey` to produce `proof.json`.
4. **Compress**: run `compress_proof.js` to get `compressed_proof.json`.
5. **Unlock**: build a transaction spending the UTxO with the redeemer encoding the compressed proof.

6.13. HOW THIS DIFFERS FROM JANUS WALLET

Janus Wallet is a production ZK wallet with staking, delegation, and a server-side proving API. This chapter's contract is intentionally stripped down to isolate the core idea.

Feature	Janus Wallet	zk-password
Public inputs	userId, hash, challenge, flags	hash only
Replay protection	On-chain challenge nonce	None (educational)
Server	Node.js + PostgreSQL	Off-chain only
Hash function	Poseidon (Circom)	Poseidon (Circom)
Curve	BLS12-381	BLS12-381
On-chain verifier	Aiken + builtins	Aiken + builtins
Staking / governance	Yes	No

The **zk-password** contract does not include replay protection. In production, a malicious observer could copy a valid proof from the blockchain and resubmit it in their own transaction before the legitimate spender. To prevent this, add a nonce or challenge to the circuit's public inputs and store an on-chain counter in the datum, as Janus Wallet does with its **challenge** and **challengeFlag** signals.

This is the complete minimal ZK contract. The next chapters will add replay protection, multi-input circuits, and a full off-chain proving workflow.

7. IMPLEMENTING OFF-CHAIN LOGIC FOR ZK IN CARDANO

7.1. THE ON-CHAIN PART IS THE SMALL PART

Chapter 6 produced a working ZK validator. Let us look at how big it actually is. After `aiken build`, the compiled validator, including the entire Groth16 verifier, the pairing equation, and the public input accumulation, occupies **570 bytes** of Plutus Core.

Everything else in a ZK application is off-chain: the circuit, the trusted setup, the witness calculation, the prover, the point compression, the datum and redeemer encoding, the transaction construction, the UTXO indexing, and the key management. That is thousands of lines of code and several hundred megabytes of artifacts, against 570 bytes on-chain.

A ZK application on Cardano is an off-chain application with a 570-byte on-chain referee. If you get the off-chain pipeline wrong, nothing works, and the on-chain error message will only ever tell you *“validation failed”*.

This chapter is about that off-chain pipeline: which software computes proofs, how to drive it step by step, how to get a proof into a redeemer with MeshJS, and what the resulting transactions actually look like and cost.

Division of Responsibilities

Responsibility	Where	Tool
Define the statement	Off-chain	Circom
Produce public parameters	Off-chain	snarkjs (ceremony)
Compute the witness	Off-chain	WASM / C++
Produce the proof	Off-chain	snarkjs / rapidsnark
Compress curve points	Off-chain	@noble/curves
Encode as Plutus Data	Off-chain	MeshJS
Build & submit transaction	Off-chain	MeshJS + provider
Verify the proof	On-chain	Aiken
Enforce spending conditions	On-chain	Aiken

Note that only two rows are on-chain, and the cryptographic one is a single function call. Cardano’s contribution to a ZK application is remarkably narrow, and, as we will measure below, remarkably cheap.

7.2. WHAT SOFTWARE COMPUTES PROOFS

There is no single “ZK prover”. There is a family of tools, and the right choice depends on where the proof must be computed: in a browser tab, on a server, or on a phone.

The Circuit Compiler

Circom is the constraint-system compiler we have been using. It takes `.circom` source and emits an R1CS file plus a witness calculator. Circom does not prove anything; it only describes the statement and computes witnesses.

```
circom --r1cs --wasm --sym --inspect \
      --prime bls12381 circuit/circuit.circom -o circuit/
```

The `--prime bls12381` flag is not optional for Cardano. Circom’s default is BN254 (Ethereum’s curve). A BN254 proof is mathematically valid and will verify with `snarkjs`, and will fail on Cardano, because Cardano’s builtins only implement BLS12-381. This mismatch is the single most common cause of “my proof verifies locally but the validator rejects it”.

Alternatives to Circom exist and are worth knowing about, though none of them currently has a Cardano verifier:

- **Noir** (Aztec): a Rust-like ZK language, much nicer to write than Circom, but its default backend (Barretenberg / UltraHonk) has no Aiken verifier.
- **Halo2** (Zcash / PSE): no trusted setup, recursive-friendly, but its verifier needs operations Cardano does not expose cheaply.
- **Circom + PLONK**: `snarkjs` can produce PLONK proofs from the same R1CS. The verifier is more complex than Groth16 and, at the time of writing, no production-grade PLONK verifier exists in Aiken. We return to this in Chapter 9.

For Cardano today, **Circom + snarkjs + Groth16 over BLS12-381** is the only fully supported path from circuit to on-chain verification. Everything in this chapter assumes it.

The Provers

Given an R1CS, a proving key, and a witness, several programs can produce the proof. They differ by orders of magnitude in speed.

Prover	Language	Runs in	Speed
snarkjs	JavaScript / WASM	Browser, Node	Baseline
rapidsnark	C++ (asm)	Server, native	5–10×
ark-circom	Rust (arkworks)	Server, native	5–10×
witnesscalc	C++	iOS, Android	10× (witness)
gnark	Go	Server	5–10×

- **snarkjs** is the reference implementation and the one to start with. It runs everywhere, needs no compilation, and is what we use in this book. Install with `npm install -g snarkjs`.
- **rapidsnark** is the production choice for server-side proving. It reads the same `.zkey` and `.wtns` files as `snarkjs` and emits an identical `proof.json`, so it is a drop-in replacement once your pipeline works. Note that `rapidsnark`'s BLS12-381 support is less mature than its BN254 support, verify this before committing.
- **ark-circom** lets you drive Circom circuits from Rust using `arkworks`. Useful if your off-chain service is already Rust, and `arkworks` has first-class BLS12-381 support.
- **witnesscalc** replaces only the witness-calculation step, which is often the bottleneck on mobile hardware.

The Supporting Cast

Package	Role
<code>circomlib</code>	Circuit primitives: Poseidon, comparators, bits
<code>circomlibjs</code>	JS reimplementations of those primitives
<code>@noble/curves</code>	Point compression to Cardano's byte format
<code>@meshsdk/core</code>	Transaction building, Plutus Data encoding
<code>@lucid-evolution/lucid</code>	Alternative to MeshJS
<code>Blockfrost</code> / <code>Koios</code> / <code>Maestro</code>	Chain queries and submission

`circomlibjs` is a trap for Cardano developers. Its `buildPoseidon` implements Poseidon over **BN254**, not BLS12-381. Calling it to compute the hash that your BLS12-381 circuit will check produces a value that is simply wrong, and the failure surfaces much later as an unsatisfiable constraint. Chapter 6 solved this by compiling a tiny helper circuit (`poseidon_hash.circom`) and reading the hash out of its witness. Do that.

7.3. THE PIPELINE, STEP BY STEP

The full pipeline has six stages. Stages 1–3 happen once per circuit. Stages 4–6 happen once per proof. Confusing the two is the second most common source of bugs.

#	Stage	Frequency	Output
1	Compile circuit	Per circuit	.r1cs, .wasm
2	Trusted setup	Per circuit	.zkey, vk.json
3	Compress the VK	Per circuit	compressed_vk.json
4	Compute witness	Per proof	.wtns
5	Prove	Per proof	proof.json, public.json
6	Compress the proof	Per proof	compressed_proof.json

Stage 1: Compile

```

circom --r1cs --wasm --sym --inspect \
      --prime bls12381 circuit/circuit.circom -o circuit/

snarkjs r1cs info circuit/circuit.r1cs

```

Read the `r1cs info` output carefully every time. It tells you the curve, the number of public inputs, and the number of constraints. The number of public inputs must equal `length(vk_ic) - 1` in your compressed verification key, and must equal the length of the list you pass to `groth16_verify` in Aiken. For our password circuit:

```

[INFO] snarkJS: Curve: bls12-381
[INFO] snarkJS: # of Wires: 219
[INFO] snarkJS: # of Constraints: 216
[INFO] snarkJS: # of Public Inputs: 1
[INFO] snarkJS: # of Private Inputs: 1
[INFO] snarkJS: # of Outputs: 0

```

Stage 2: Trusted Setup

Phase 1 (Powers of Tau) is circuit-independent and reusable. Phase 2 is bound to this specific R1CS.

```

# Phase 1: universal, reusable across circuits
snarkjs powersoftau new bls12381 12 pot12_0000.ptau -v
snarkjs powersoftau contribute pot12_0000.ptau pot12_0001.ptau \
      --name="dev" -e="some entropy"
snarkjs powersoftau beacon pot12_0001.ptau pot12_beacon.ptau \
      00000000000000000000000000000000 10 -n="Dev beacon"
snarkjs powersoftau prepare phase2 \
      pot12_beacon.ptau pot12_final.ptau -v

# Phase 2: specific to circuit.r1cs
snarkjs groth16 setup \
      circuit/circuit.r1cs pot12_final.ptau zkey_0000.zkey
snarkjs zkey contribute zkey_0000.zkey zkey_0001.zkey \

```



```
--name="dev" -e="more entropy"
snarkjs zkey beacon zkey_0001.zkey zkey_final.zkey \
    00000000000000000000000000000000 10 -n="Final beacon"
snarkjs zkey export verificationkey \
    zkey_final.zkey verification_key.json
```

The power 12 means $2^{12} = 4096$ maximum constraints. Choose the smallest power that exceeds your constraint count: the `.ptau` and `.zkey` file sizes grow linearly with 2^{power} , and so does the proving time. This matters enormously in practice; we will see in Chapter 8 that a circuit needing 2^{20} turns a 2 MB proving key into a multi-gigabyte one.

The proving key is not a secret, but it is not small either. Stage 2 is the point at which a ZK application acquires a distribution problem: every prover needs `zkey_final.zkey` and `circuit.wasm`, and every verifier needs to agree on the same `compressed_vk.json`.

Stage 3: Compress the Verification Key

Cardano's BLS12-381 builtins accept only compressed points: 48 bytes for G1, 96 bytes for G2. `snarkjs` emits uncompressed affine coordinates. The `compress_vk.js` script from Chapter 6 does the conversion:

```
node circuit/compress_vk.js
```

This is a per-circuit operation. The output is the value you will embed in the datum (or hard-code in the validator, or store in a reference input, see Section 7.9).

Stage 4: Compute the Witness

For our circuit the witness needs the password as a field element and its Poseidon hash over BLS12-381:

```
node circuit/gen_input.js 123456

password : 123456
hash      : 941543474650874718122864576426741288983580381
           5708024285868314342086739058356
written   : circuit/input.json
```

`gen_input.js` is the only place in the entire pipeline that touches the plaintext password. Everything downstream operates on field elements and curve points. When you audit a ZK application, this is the file to audit first.

Stage 5: Prove

```
snarkjs groth16 fullprove \
  circuit/input.json \
  circuit/circuit_js/circuit.wasm \
  zkey_final.zkey \
  proof.json public.json

snarkjs groth16 verify verification_key.json public.json proof.json
```

Always run the off-chain `verify` before building a transaction. It costs milliseconds and distinguishes “my proof is wrong” from “my encoding is wrong”, two failures that look identical from on-chain.

Stage 6: Compress the Proof

```
node circuit/compress_proof.js
```

The result is three hex strings. For our worked example:

```
{
  "pi_a": "b15f6da17265cbb596036075e87ddfe62d210a92c65cb30a
          c65375659ef26511c50b55b989abc6da22f05495fca74380",
  "pi_b": "ac6f06b66982fc793de1a1d393b66f80f967cf4f984bc18a
          c9e034a618d732f5a9dd9c632da14178b54584a682f80708
          08437cb5f80c0559f9cddf71f7edbddd4853249ba637ec3a
          59950eff969e99a2d4574a6f229a753cdb2d5674fcf060cf",
  "pi_c": "8d6da5b26036b096441e48b6c490f56c4a4f9148a881b0e5
          0c556626a4cef37d672c0e8d7c74fc0b2214e0bf63f36f8a"
}
```

(Line breaks added for the page; the real file has one string per field.) That is **192 bytes** of proof (48 + 96 + 48), regardless of how complicated the statement was. This constant size is Groth16’s defining property and the reason it suits a blockchain.

7.4. WHERE SHOULD THE PROVER RUN?

This is an architectural decision, not a technical one, and it determines whether your application actually provides privacy. There are three options.

Architecture A: Client-Side Proving

The witness and the proof are computed in the user’s browser or app. The secret never leaves the device.

[user's browser]				[chain]
secret --> witness --> proof -----> validator				
(wasm)	(snarkjs)			(Aiken)

- **Privacy:** maximal. Nobody but the user ever sees the secret.
- **Cost:** the user downloads the .zkey (megabytes to gigabytes) and pays the proving time.
- **Verdict:** the only architecture that honestly delivers zero-knowledge. Use it whenever the circuit is small enough.

Architecture B: Server-Side Proving

The user sends the secret to a server, which computes the proof and returns it (or builds the whole transaction). This is what Janus Wallet¹ does: a Node.js service with `circom` installed, PostgreSQL for user and circuit records, and Blockfrost for submission.

[browser]		[prover service]		[chain]
secret -----> witness --> proof -----> validator				
(TLS)		(rapidsnark)		

- **Privacy:** *none against the server*. The server sees the password. The proof is still zero-knowledge with respect to the blockchain and every other observer, but the operator is fully trusted.
- **Cost:** fast proving, no large client download.
- **Verdict:** legitimate for onboarding and UX, provided you say so plainly. It is a custodial-style trust assumption wearing a zero-knowledge badge.

“The proof is zero-knowledge” and “the system is zero-knowledge” are different claims. A server-side prover satisfies the first and violates the second. Be explicit about which one you are offering.

Architecture C: Hybrid

The witness is computed client-side; the proof is computed server-side from the witness. This does not help: for most circuits the witness contains the private signals verbatim, so the server still learns the secret. The variant that *does* help is delegating only the transaction *submission*, a relayer. The relayer sees the proof (public anyway) and pays the fee, but never sees the witness. We use exactly this pattern in Chapter 8.

7.5. ENCODING PROOFS AS PLUTUS DATA

The bridge between snarkjs and Aiken is Plutus Data. Get the shape right and everything works; get the field order wrong and you receive an unhelpful validation failure.

¹<https://github.com/leobel1/janus-wallet>

The Mapping Rules

Aiken	Plutus Data	MeshJS
Int	I integer	number / bigint
ByteArray	B bytes	hex string
List<a>	List	Array
record / constructor	Constr <i>n</i>	mConStrn([...])
Option.Some(x)	Constr 0	mConStr0([x])
Option.None	Constr 1	mConStr1([])

Three rules govern the translation:

1. **Field order is positional.** Aiken records serialise their fields in declaration order. The Plutus Data Constr carries no field names, so `mConStr0` arguments must appear in exactly the order the Aiken type declares them.
2. **Single-constructor records use alternative 0.**
3. **Opaque types cannot cross the boundary.** As we discovered in Chapter 6, `Scalar` is opaque; the datum must carry a plain `Int` and the validator converts it with `scalar.new`.

Our Types, Encoded

Recall the Aiken declarations:

```
pub type VerificationKey {
  alpha: ByteArray,      // G1: 48 bytes
  beta:  ByteArray,      // G2: 96 bytes
  gamma: ByteArray,      // G2: 96 bytes
  delta: ByteArray,      // G2: 96 bytes
  ic:    List<ByteArray>, // G1 points
}

pub type Proof {
  pi_a: ByteArray, pi_b: ByteArray, pi_c: ByteArray,
}

pub type Datum    { credential_hash: Int, vk: VerificationKey }
pub type Redeemer { proof: Proof }
```

And the corresponding MeshJS encoding:

```
import { mConStr0 } from "@meshsdk/core";
import compressedVk from "./compressed_vk.json";
import compressedProof from "./compressed_proof.json";
import publicSignals from "./public.json";
```

```
// VerificationKey: five fields, in declaration order
const vkData = mConStr0([
  compressedVk.vk_alpha1,
  compressedVk.vk_beta2,
  compressedVk.vk_gamma2,
  compressedVk.vk_delta2,
  compressedVk.vk_ic,          // List<ByteArray> -> JS Array
]);

// Datum { credential_hash: Int, vk: VerificationKey }
// public.json holds a decimal string; it exceeds 2^53, so BigInt
// is mandatory. parseInt() here silently corrupts the value.
const datumData = mConStr0([
  BigInt(publicSignals[0]),
  vkData,
]);

// Proof, then Redeemer { proof: Proof }
const proofData = mConStr0([
  compressedProof.pi_a,
  compressedProof.pi_b,
  compressedProof.pi_c,
]);

const redeemerData = mConStr0([proofData]);
```

The `credential_hash` is a 255-bit field element. JavaScript's `number` holds 53 bits of integer precision. Using `parseInt` or `Number` on it produces a nearby-but-wrong value, the datum hash changes, and the validator fails on a value that *looks* right in the console. Always `BigInt`.

Sanity-Check the Encoding Before Submitting

You can verify the datum round-trips without spending anything:

```
import { serializeData, deserializeDatum } from "@meshsdk/core";

const cbor = serializeData(datumData, "Mesh");
console.log("datum cbor:", cbor);
console.log("round-trip:", deserializeDatum(cbor));
```

Compare the CBOR against what aiken blueprint reports for the datum schema. A mismatch here is a mismatch on-chain.

7.6. BUILDING THE TRANSACTIONS WITH MESHJS

We now have all the pieces. Let us build the two transactions: one to lock ADA behind the ZK lock, one to unlock it with a proof.

Setup: common.ts

```
npm install @meshsdk/core @meshsdk/core-csl
```

```
import fs from "node:fs";
import {
  BlockfrostProvider, MeshTxBuilder, MeshWallet,
  serializePlutusScript, UTxO,
} from "@meshsdk/core";
import { applyParamsToScript } from "@meshsdk/core-csl";
import blueprint from "../plutus.json";

const projectId = process.env.BLOCKFROST_PROJECT_ID;
if (!projectId) throw new Error("Missing BLOCKFROST_PROJECT_ID");

export const provider = new BlockfrostProvider(projectId);

export const wallet = new MeshWallet({
  networkId: 0, // 0 = preprod/preview, 1 = mainnet
  fetcher: provider,
  submitter: provider,
  key: {
    type: "root",
    bech32: fs.readFileSync("me.sk").toString(),
  },
});

export function getScript() {
  const scriptCbor = applyParamsToScript(
    blueprint.validators[0].compiledCode, []
  );
  const scriptAddr = serializePlutusScript(
    { code: scriptCbor, version: "V3" }
  ).address;
  return { scriptCbor, scriptAddr };
}

export function getTxBuilder() {
  return new MeshTxBuilder({
    fetcher: provider,
    submitter: provider,
    evaluator: provider,
  });
}

export async function getUtxoByTxHash(
  txHash: string
): Promise<UTxO> {
  const utxos = await provider.fetchUTxOs(txHash);
  if (utxos.length === 0) throw new Error("UTxO not found");
  return utxos[0];
}
```

Passing evaluator lets MeshJS ask the provider to evaluate the script and fill in the execution-unit budget automatically, instead of you guessing it.

lock.ts: Locking Behind the ZK Lock

```
import { Asset, mConStr0 } from "@meshsdk/core";
import { getScript, getTxBuilder, wallet } from "../common";
import compressedVk from "../compressed_vk.json";
import publicSignals from "../public.json";

async function main() {
  const assets: Asset[] =
    [{ unit: "lovelace", quantity: "5000000" }];

  const utxos = await wallet.getUtxos();
  const walletAddress = (await wallet.getUsedAddresses())[0];
  const { scriptAddr } = getScript();

  const vkData = mConStr0([
    compressedVk.vk_alpha1, compressedVk.vk_beta2,
    compressedVk.vk_gamma2, compressedVk.vk_delta2,
    compressedVk.vk_ic,
  ]);

  const datum = mConStr0([BigInt(publicSignals[0]), vkData]);

  const txBuilder = getTxBuilder();
  await txBuilder
    .txOut(scriptAddr, assets)
    .txOutInlineDatumValue(datum)
    .changeAddress(walletAddress)
    .selectUtxosFrom(utxos)
    .complete();

  const signedTx = await wallet.signTx(txBuilder.txHex);
  const txHash = await wallet.submitTx(signedTx);
  console.log("locked at:", txHash);
}

main();
```

We use `txOutInlineDatumValue`, not `txOutDatumHashValue`. With an inline datum the verification key and the Poseidon hash live directly in the UTxO, so any spender can read them from the chain without a side channel. With a datum hash the spender would have to obtain the preimage from somewhere else: extra infrastructure for no privacy benefit, since neither value is secret.

Storing the verification key in the datum makes each locked UTxO self-describing, at the cost of roughly 430 bytes of on-chain data per UTxO. Section 7.9 discusses the alternative: bake the VK into the validator as a compile-time parameter.

unlock.ts: Spending with a Proof

```

import { mConStr0 } from "@meshsdk/core";
import {
  getScript, getTxBuilder, getUtxoByTxHash, wallet,
} from "../common";
import compressedProof from "../compressed_proof.json";

async function main() {
  const lockTxHash = process.argv[2];

  const utxos = await wallet.getUtxos();
  const walletAddress = (await wallet.getUsedAddresses())[0];
  const collateral = (await wallet.getCollateral())[0];

  const { scriptCbor } = getScript();
  const scriptUtxo = await getUtxoByTxHash(lockTxHash);

  // Redeemer { proof: Proof { pi_a, pi_b, pi_c } }
  const redeemer = mConStr0([
    mConStr0([
      compressedProof.pi_a,
      compressedProof.pi_b,
      compressedProof.pi_c,
    ]),
  ]);

  const txBuilder = getTxBuilder();
  await txBuilder
    .spendingPlutusScript("V3")
    .txIn(
      scriptUtxo.input.txHash,
      scriptUtxo.input.outputIndex,
      scriptUtxo.output.amount,
      scriptUtxo.output.address
    )
    .txInScript(scriptCbor)
    .txInRedeemerValue(redeemer)
    .txInInlineDatumPresent()
    .txInCollateral(
      collateral.input.txHash,
      collateral.input.outputIndex,
      collateral.output.amount,
      collateral.output.address
    )
    .changeAddress(walletAddress)
    .selectUtxosFrom(utxos)
    .complete();

  const signedTx = await wallet.signTx(txBuilder.txHex);
  const txHash = await wallet.submitTx(signedTx);
  console.log("unlocked at:", txHash);
}

```



```
main();
```

Three details deserve attention.

- `txInInlineDatumPresent()` tells the builder the datum is already on-chain. If the lock used a datum hash you would call `txInDatumValue(datum)` and supply the preimage instead.
- `txInCollateral` is mandatory for any Plutus spend. If the script fails, the collateral is forfeited, which is why you test with `aiken check` first, then on preprod, then on mainnet.
- There is no `requiredSignerHash`. Our validator does not check signatures at all: possession of a valid proof *is* the authorisation. That is the whole point, and also, as Chapter 6 warned, why replay protection matters (Section 7.9).

Running It

```
export BLOCKFROST_PROJECT_ID=preprodXXXXXXXXXXXXXXXXXX
npx tsx offchain/lock.ts
npx tsx offchain/unlock.ts <lock-tx-hash>
```

7.7. WHAT THE TRANSACTIONS ACTUALLY COST

Abstract talk about efficiency is worth less than measurement. Aiken reports execution units for every test, so we can read the real cost of our verifier directly:

```
aiken check
```

```
PASS zk_password/password_tests :: groth16_verify_valid_proof
    mem: 34219, cpu: 2005373690
```

Compare against the live mainnet protocol parameters (epoch 646, Conway era, protocol version 11):

Resource	Used	Budget
CPU steps	2,005,373,690	10,000,000,000
Memory units	34,219	16,500,000
Script size (bytes)	570	16,384 (whole tx)

So a Groth16 verification consumes **20.1% of the CPU budget** and **0.21% of the memory budget**. At the current prices, 0.0000721 lovelace per CPU step and 0.0577 lovelace per memory unit, the script execution costs:

$$2,005,373,690 \times 0.0000721 + 34,219 \times 0.0577 \approx 146,562 \text{ lovelace} \approx \mathbf{0.147 \text{ ADA}}$$

Verifying a zero-knowledge proof on Cardano costs about 0.15 ADA and one fifth of a transaction’s CPU budget, and that figure does not depend on how complicated the circuit was. Groth16 verification is three point decompressions and four pairings, whether the statement had 216 constraints or 216,000.

This constancy is the single most important economic fact about ZK on Cardano, and we will lean on it heavily in Chapter 8. Its immediate corollary is a hard ceiling:

$$\left\lfloor \frac{10,000,000,000}{2,005,373,690} \right\rfloor = 4 \text{ Groth16 verifications per transaction}$$

You cannot batch five ZK spends into one transaction. The memory budget would allow 482 of them; the CPU budget allows four.

7.8. GENERATING THE OFF-CHAIN CODE: AIKEN-ZK

Writing all of this by hand, once, is educational. Writing it for every circuit is toil. The **aiken-zk** tool from Eryx², funded by Project Catalyst Fund 13, automates most of it.

The idea is a language extension: you annotate an Aiken validator with an **offchain** block describing a computation, and the tool emits the Circom circuit, the standard Aiken validator containing the Groth16 verifier, and the off-chain proving glue.

The Workflow

```
aiken-zk new my_project

# transform annotated Aiken into plain Aiken + circuit + vk
aiken-zk build code_with_offchain.ak aiken_code.ak

# produce a proof as an Aiken literal (for tests)
aiken-zk prove aiken output.circom verification_key.zkey \
    inputs.json proof.ak

# produce a proof as a ready-made MeshJS redeemer
aiken-zk prove meshjs output.circom verification_key.zkey \
    inputs.json zk_redeemer.ts
```

That last command is the interesting one for this chapter: it emits the TypeScript redeemer object directly, replacing Stage 6 and the hand-written `mConStr0` nesting of Section 7.5.

²<https://github.com/eryxcoop/cardano-zk-aiken>

The offchain Vocabulary

Statement	Proves
<code>addition(x, y, z)</code>	$x + y = z$
<code>subtraction(x, y, z)</code>	$x - y = z$
<code>multiplication(x, y, z)</code>	$x \cdot y = z$
<code>fibonacci(x, y, t, z)</code>	t -th Fibonacci term is z
<code>if(x, y, z, w)</code>	$y = z$ if $x = 1$, else $y = w$
<code>assert_eq(x, y)</code>	$x = y$
<code>sha256(t, A, B)</code>	$\text{sha256}(A) = B$
<code>poseidon(t, A, x)</code>	$\text{Poseidon}(A) = x$
<code>polynomial_evaluations</code>	polynomial evaluation
<code>merkle_tree_checker</code>	Merkle inclusion path
<code>custom("...circom", [...])</code>	an arbitrary Circom component

Arguments carry visibility modifiers, so `offchain addition(priv x, pub y, z)` makes `x` a private witness and `y` a public input. The `custom` escape hatch is what makes the tool genuinely useful: you can drop in any Circom component and still get the verifier and redeemer generated.

Current Limitations

The project is young, and its README is candid about the edges:

- Only **one** `offchain` statement per Aiken file.
- `fibonacci`'s length parameter must be a literal.
- List arguments cannot be literals.
- **The `merkle_tree_checker` hash function is mocked: “do not use in production”.**

That last bullet is not an incidental bug. It is the same wall we will hit throughout Chapter 8, and it has a specific cause: Cardano has no ZK-friendly hash primitive. We will measure exactly how expensive that absence is.

Use `aiken-zk` to bootstrap and to learn the shape of the generated code, but read what it emits. A tool that hides the Circom layer will eventually hide a curve mismatch or a public-input ordering bug from you, and those are precisely the bugs that cost collateral.

7.9. OPERATIONAL CONCERNS

A working pipeline is not a working product. Five issues bite in production.

1. Replay Protection

Our validator accepts any valid proof for the hash in the datum. A proof is public data sitting in a submitted transaction. An observer watching the mempool can copy the proof and submit their own transaction paying themselves.

The fix is to bind the proof to something unique. Add a public input the validator can independently check:

- **A nonce in the datum.** The validator requires the continuing output's datum to carry `nonce + 1`, and the circuit takes the nonce as a public input. Janus Wallet uses exactly this, with `challenge` and `challengeFlag` signals.
- **The recipient address.** Make the payment destination a public input. A stolen proof can then only pay the original recipient, which removes the attacker's incentive.
- **The spent output reference.** Bind the proof to the specific UTxO being consumed, which is unique by construction.

Any ZK validator that authorises value transfer on the strength of a proof alone, with no binding to the transaction, is replayable. This is not a Cardano quirk; it is true on every chain. Treat a proof as a bearer instrument.

2. Where to Put the Verification Key

Three options, with real trade-offs:

Placement	Cost	Upgradeable
Inline datum	~430 bytes per UTxO	Yes, per UTxO
Compile-time param	Larger script, once	No (new address)
Reference input	15 lovelace/byte fee	Yes, centrally

Baking the VK into the script with `applyParamsToScript` is the cheapest per transaction and the most rigid: changing the circuit changes the script hash, hence the address, and funds at the old address need migrating. This is the trap Janus Wallet documents, a password change migrates the address, and funds sent to the old one are unrecoverable.

The reference-input approach stores the VK in a UTxO consulted read-only by many transactions. With `minFeeRefScriptCoinsPerByte` currently at 15, our 570-byte validator costs $570 \times 15 = 8,550$ lovelace (≈ 0.009 ADA) per transaction when referenced, cheaper than including the script inline once you have more than a handful of spends.

3. Distributing the Ceremony Artifacts

Every prover needs `circuit.wasm` and `zkey_final.zkey`. These are not secrets, but they are integrity-critical: a substituted `zkey` lets an attacker forge proofs that your on-chain VK will happily accept only if the VK was substituted too, so pin both. Publish the hashes, ideally on-chain alongside the VK, and have clients check them.

4. Budget Estimation and Collateral

Let the provider’s evaluator compute execution units rather than hard-coding them. Hard-coded budgets from a tutorial will be wrong after the next cost-model update, and an underestimate means a failed script and forfeited collateral. Note that `maxCollateralInputs` is 3 and `collateralPercentage` is 150, so you need a clean, ADA-only UTxO of sufficient size available at all times.

5. Indexing

Nothing in Cardano tells you “here are the UTxOs at my script address with a datum matching this hash”. You need to query and filter. Blockfrost’s `/addresses/{addr}/utxos` plus datum decoding works for small applications; anything larger wants Kupo, Ogmios, or Carp maintaining an index. Chapter 8 shows what happens when the off-chain indexer must reconstruct cryptographic state, not just find UTxOs.

7.10. FAILURE MODES AND HOW TO RECOGNISE THEM

Almost every off-chain ZK bug on Cardano presents as the same unhelpful symptom. This table is the debugging shortcut this chapter exists to provide.

Symptom	Cause
snarkjs verifies, chain rejects	Circuit compiled for BN254. Recompile with <code>--prime bls12381</code> .
<code>decompress</code> fails / script errors immediately	Uncompressed points in datum or redeemer. Run <code>compress_vk.js</code> and <code>compress_proof.js</code> .
Constraint unsatisfiable during <code>fullprove</code>	Hash computed with <code>circomlibjs</code> (BN254) instead of the BLS12-381 helper circuit.
I caught an opaque type trying to escape	Scalar in a datum or redeemer. Use <code>Int</code> and <code>scalar.new</code> inside the validator.
Verification fails with a correct-looking hash	<code>parseInt/Number</code> truncated the 255-bit field element. Use <code>BigInt</code> .
Fails only for some public inputs	<code>vk_ic</code> order wrong. IC_0 is the constant term; IC_{i+1} pairs with public input i .
<code>ExUnits</code> exceeded	More than four Groth16 verifications, or on-chain field arithmetic. See Chapter 8.
Datum mismatch at spend time	Field order differs between the Aiken <code>type</code> and the <code>mConStr0</code> array.

Notice how many of these are *encoding* bugs rather than cryptographic ones. The mathematics is handled by libraries that are correct. The bugs live in the seams: field choice, point format, integer width, field order. Build a round-trip test for your encoding before you build a transaction.

7.11. CHAPTER SUMMARY

- The on-chain component of a ZK application on Cardano is tiny: 570 bytes of Plutus Core for a complete Groth16 verifier.
- Circom + snarkjs + Groth16 over BLS12-381 is the only fully supported path today. `--prime bls12381` is not optional.
- Six pipeline stages: three per circuit (compile, setup, compress VK), three per proof (witness, prove, compress proof).
- Where the prover runs decides whether the system is actually private. Client-side proving is the only architecture that delivers zero-knowledge against the operator.
- Plutus Data encoding is positional. `BigInt`, correct field order, compressed points.
- MeshJS builds both transactions in about thirty lines each: `txOutInlineDatumValue` to lock, `spendingPlutusScript("V3")` with `txInRedeemerValue` to unlock.
- A measured Groth16 verification costs 2,005,373,690 CPU steps and 34,219 memory units, 20% of the CPU budget, ≈ 0.147 ADA, and *constant* in circuit size. Four verifications per transaction is the hard ceiling.
- `aiken-zk` generates most of this from an `offchain` annotation, including the MeshJS redeemer, with the notable caveat that its Merkle tree hash is mocked.
- Proofs are bearer instruments. Bind them to a nonce, a recipient, or an output reference, or they are replayable.

The next chapter takes everything in this one and points it at a hard target: a Tornado Cash-style mixer. We will find that the verification side works fine, and that the problem lies somewhere else entirely.

8. TORNADO CASH ON CARDANO: A CASE STUDY

8.1. SCOPE AND PURPOSE

This chapter is an academic case study. Its purpose is to take the most demanding well-known ZK application and work out, concretely, whether Cardano can host it today. The answer turns out to be *no*, and the reasons why teach more about the platform's real limits than any working example could. No deployable implementation is provided. Nothing in this chapter is deployed to any network, and no complete circuit or validator set is given. The code fragments are design sketches for analysis.

There is a second reason for the disclaimer. Tornado Cash was sanctioned by the US Office of Foreign Assets Control in August 2022, and a developer was subsequently prosecuted over its operation. Whatever one's view of those actions, which were and remain contested, deploying a mixer carries legal exposure that has nothing to do with whether the cryptography works. This chapter analyses a design. It does not recommend building one.

What we are actually after is the engineering question, which is genuinely interesting and has a genuinely surprising answer:

Cardano can verify a zero-knowledge proof for about 0.15 ADA and 20% of a transaction's CPU budget, at a cost that does not depend on the circuit's complexity. Why, then, can it not host an application whose on-chain component is one proof verification?

Because verifying a proof and *maintaining the state the proof is about* are completely different problems, and Cardano is excellent at the first and poor at the second. This chapter measures exactly how poor.

8.2. WHAT TORNADO CASH ACTUALLY DOES

Strip away the politics and Tornado Cash is a clean cryptographic construction: a **shielded pool** built from three ingredients.

The Deposit

A depositor generates two random 248-bit values, a nullifier and a **secret**, and computes a **commitment**:

$$C = H(\text{nullifier}, \text{secret})$$

They send a fixed amount (1 ETH, or 10, or 100) to the pool contract along with C . The contract appends C as a leaf to an append-only **incremental Merkle tree** of

depth 20 and stores the new root. The pair (`nullifier`, `secret`) is the depositor's *note*; losing it loses the funds.

Crucially, the contract does not record who deposited which commitment beyond what the transaction itself reveals. The commitment is just a hash.

The Withdrawal

Later, and ideally much later, the depositor proves in zero knowledge that:

1. they know a (`nullifier`, `secret`) pair whose commitment C is a leaf of the tree with root R , *without revealing which leaf*; and
2. the public value `nullifierHash` = $H(\text{nullifier})$ is correctly derived from that same nullifier.

The contract checks the proof, checks that R is a recent known root, checks that `nullifierHash` has not been seen before, records it, and pays the denomination to a recipient address supplied as a public input.

Why the Three Pieces Are Each Necessary

Piece	What breaks without it
Merkle tree	No way to prove membership in the deposit set without naming your deposit; the link is exposed.
Nullifier	The same note could be withdrawn repeatedly; the pool is drained.
Fixed denomination	Amounts become fingerprints. A deposit and a withdrawal of 3.7194 ADA are trivially linked.

The anonymity set is the number of deposits in the tree that could plausibly be yours. A pool with 3 deposits provides essentially no privacy; Tornado's largest pools held tens of thousands. **Anonymity is a function of pool throughput**, and this will matter enormously later.

The Relayer

There is a bootstrapping problem: to withdraw, you need a transaction, and a transaction needs a fee, and a fee needs funds, and if you pay the fee from an address linked to your deposit, you have undone the entire construction. Tornado solves this with **relayers**: third parties who submit the withdrawal transaction and pay the fee, taking a cut from the withdrawn amount. The relayer learns the proof and the recipient but never the note.

8.3. SKETCHING IT IN EUTXO

Let us design it for Cardano properly, giving the eUTxO model every advantage, before looking for problems.

The Pool State

Ethereum's contract has mutable storage. Cardano has UTxOs. The natural translation is a single **pool UTxO** carrying the tree state in its datum, consumed and recreated by every deposit:

```
pub type PoolDatum {
  root:      ByteArray,      // current Merkle root
  next_index: Int,           // number of leaves so far
  filled:     List<ByteArray>, // rightmost node per level
  denom:      Int,           // fixed denomination, lovelace
}
```

The `filled` list is the standard incremental-Merkle-tree trick: to append a leaf and recompute the root you do not need the whole tree, only the rightmost filled node at each of the 20 levels, plus a table of precomputed "zero" subtree hashes. That is 20 hashes per insertion instead of a million.

The Nullifier Set: Where Cardano Wins

Ethereum stores spent nullifiers in a mapping(`bytes32 => bool`). On Cardano, tracking a growing set is normally painful. But there is an unusually elegant option here, and it is worth pausing on because it is genuinely better than the Ethereum version.

Mint a token whose asset name is the nullifier hash.

The Cardano ledger enforces, as a consensus rule, that a given (policy id, asset name) pair with quantity 1 cannot be minted twice under a policy that only ever mints one. So the minting policy becomes the double-spend check, and it is enforced by the ledger rather than by our script:

```
validator nullifier_policy {
  mint(redeemer: WithdrawRedeemer, policy_id: PolicyId,
    tx: Transaction) {
    expect [Pair(asset_name, 1)] =
      tx.mint |> assets.tokens(policy_id) |> dict.to_pairs()

    // The minted asset name IS the nullifier hash, so uniqueness
    // is a ledger invariant, not something we have to check.
    and {
      asset_name == redeemer.nullifier_hash,
      verify_withdraw_proof(redeemer),
    }
  }
}
```

```

    }
}

```

This is a real advantage of the eUTxO model. On Ethereum, double-withdrawal prevention costs an `SSTORE` and depends on the contract being correct. On Cardano it costs a token mint and depends on the *ledger* being correct. No script bug can permit a double withdrawal.

So far Cardano is doing well. The trouble is elsewhere.

8.4. THE WITHDRAWAL CIRCUIT

The circuit is a direct adaptation of Tornado's, with Poseidon in place of the original Pedersen/MiMC pair; Poseidon is what circomlib gives us over BLS12-381, and it is what Chapter 6 already used.

```

pragma circom 2.1.6;

include "../node_modules/circomlib/circuits/poseidon.circom";
include "../node_modules/circomlib/circuits/bitify.circom";

// One level of a Merkle path. `sel` is the path bit: 0 means our
// node is on the left, 1 means on the right.
template MerkleLevel() {
    signal input  cur;
    signal input  sibling;
    signal input  sel;
    signal output out;

    sel * (sel - 1) === 0;    // force sel to be a bit

    signal left;
    signal right;
    left  <== cur + sel * (sibling - cur);
    right <== sibling + sel * (cur - sibling);

    component h = Poseidon(2);
    h.inputs[0] <== left;
    h.inputs[1] <== right;
    out <== h.out;
}

template Withdraw(levels) {
    // ---- public inputs (checked by the Aiken validator) ----
    signal input root;
    signal input nullifierHash;
    signal input recipient;    // payment credential hash
    signal input fee;          // relayer fee in lovelace

    // ---- private witness (never leaves the prover) ----

```

```

    signal input nullifier;
    signal input secret;
    signal input pathElements[levels];
    signal input pathIndices[levels];

    // 1. commitment = Poseidon(nullifier, secret)
    component commit = Poseidon(2);
    commit.inputs[0] <== nullifier;
    commit.inputs[1] <== secret;

    // 2. nullifierHash = Poseidon(nullifier)
    component nh = Poseidon(1);
    nh.inputs[0] <== nullifier;
    nh.out == nullifierHash;

    // 3. the commitment is a leaf under `root`
    component path[levels];
    signal cur[levels + 1];
    cur[0] <== commit.out;

    for (var i = 0; i < levels; i++) {
        path[i] = MerkleLevel();
        path[i].cur <== cur[i];
        path[i].sibling <== pathElements[i];
        path[i].sel <== pathIndices[i];
        cur[i + 1] <== path[i].out;
    }

    cur[levels] == root;

    // 4. Bind recipient and fee into the constraint system so a
    //     relayer cannot redirect the payment. Squaring is the
    //     standard Circom idiom for "this signal must be used".
    signal recipientSq;
    signal feeSq;
    recipientSq <== recipient * recipient;
    feeSq <== fee * fee;
}

component main {public [root, nullifierHash, recipient, fee]}
    = Withdraw(20);

```

Two Cardano-specific notes. First, `recipient` is a payment credential hash (28 bytes, 224 bits), which fits comfortably in a 255-bit BLS12-381 field element, so the address can be a single public input. Second, binding `recipient` and `fee` into the circuit is the replay protection from Section 7.9: a stolen proof can only ever pay the original recipient.

This circuit is correct. Compiled for BLS12-381 it produces a proof that Cardano's builtins can verify in 2.0 billion CPU steps, exactly as in Chapter 6. **The withdrawal side of Tornado Cash is not the problem.**

8.5. WALL ONE: THERE IS NO ON-CHAIN POSEIDON

The problem is the *deposit*. To append a leaf, the pool validator must recompute the Merkle root, and that means computing Poseidon *on-chain*, in Aiken, 20 times.

Cardano has no Poseidon builtin. The available hash builtins are `sha2_256`, `sha3_256`, `blake2b_224`, `blake2b_256`, `keccak_256` and `ripemd_160`; all excellent, none of them ZK-friendly. So Poseidon would have to be written in Aiken, from field arithmetic.

What Field Arithmetic Costs, Measured

Aiken's `Scalar` type is not a native field element. Looking at the `stdlib` source:

```
pub fn mul(left: Scalar, right: Scalar) -> Scalar {
  Scalar(left.integer * right.integer % field_prime)
}
```

That is arbitrary-precision integer multiplication followed by a 255-bit modulo, interpreted by the Plutus CEK machine. There is no BLS12-381 scalar arithmetic builtin. So let us measure it rather than guess.

The benchmark file in the repository, `lib/zk_password/bench_onchain.ak`, runs tail-recursive loops of 1000 iterations with matching `nop` baselines, so subtracting the baseline and dividing by 1000 gives the marginal cost of one operation:

```
aiken check -m bench
```

Operation	CPU steps	Mem units
<code>acc + 3</code> (small ints)	197,240	602
<code>acc % p</code>	250,615	604
<code>scalar.add</code>	417,083	1,009
<code>scalar.mul</code>	421,644	1,011
<code>raw (a + b) % p</code>	417,083	1,009
<code>raw a * b % p</code>	430,929	1,012
<code>blake2b_256</code> (64-byte input)	398,720	813
<code>sha2_256</code> (64-byte input)	581,923	813

Three things in this table matter more than the headline numbers.

1. **The `Scalar` wrapper is not the problem.** `Raw a * b % p` on plain integers costs the same as `scalar.mul`. Bypassing the `stdlib` buys nothing.
2. **Operand width is not the problem either.** A bare small-integer `acc + 3` already costs 197,240 CPU steps. The 255-bit modular multiply costs only about twice that.

3. **Therefore the cost is the interpreter, not the arithmetic.** Roughly 200,000 CPU steps and 600 memory units are the price of *any* elementary operation in a Plutus script. The numbers are dominated by CEK machine overhead per reduction step.

This is the most consequential fact in the chapter. Because the cost floor is *per-operation* rather than *per-bit*, no optimisation helps. Montgomery form, lazy reduction, batched inversion, a hand-tuned UPLC implementation; all of them reduce the *width* of the arithmetic, and width is not what you are paying for. You are paying about 2×10^5 CPU steps per operation, and Poseidon needs about 1.4×10^3 of them.

Poseidon, Costed

Poseidon over BLS12-381 with $t = 3$ (the 2-to-1 compression a binary Merkle tree needs) uses circomlib's standard parameters: 8 full rounds and 57 partial rounds, with an x^5 S-box.

Per round	Mults	Adds	Rounds
Full: 3 S-boxes (3×), MDS (9×)	18	9	8
Partial: 1 S-box (3×), MDS (9×)	12	9	57
Total	828	585	

At the measured rates:

$$828 \times 430,929 + 585 \times 417,083 = 600,802,860 \text{ CPU steps}$$

$$828 \times 1,012 + 585 \times 1,009 = 1,428,386 \text{ memory units}$$

So **one** on-chain Poseidon compression costs 6.0% of the transaction CPU budget and 8.7% of the memory budget. About 0.126 ADA per hash.

The Verdict on the Deposit

A depth-20 insertion needs 20 of them:

Depth-20 Merkle insertion	CPU	Memory
Poseidon in Aiken (estimated)	12,016,057,195	28,567,718
Transaction budget	10,000,000,000	16,500,000
Utilisation	120%	173%
blake2b_256 builtin, 20 levels	7,974,399	16,254
Utilisation	0.08%	0.10%

A depth-20 Poseidon insertion exceeds the CPU budget by 20% and the memory budget by 73%. It does not fit, and it does not nearly fit. Working backwards from the memory budget, the deepest tree that fits is **11 levels**, an anonymity set of 2,048, consuming essentially the entire transaction budget, leaving nothing for the rest of the validator.

The same operation with the `blake2b_256` builtin costs 0.08% of the CPU budget. **Poseidon on-chain is roughly 1,500 times more expensive than blake2b on CPU and 1,750 times on memory.**

The Poseidon figures are estimates built from measured primitives, and they are *lower bounds*. They count only field multiplications and additions. A real Aiken implementation also pays for traversing the round-constant lists, indexing the MDS matrix, and constructing intermediate lists; all at the same $\approx 2 \times 10^5$ steps per operation. The true cost is higher, not lower.

This is also precisely why `aiken-zk`'s `merkle_tree_checker` ships with a mocked hash function and a “do not use in production” warning (Section 7.8). It is not an oversight. There is nothing to put there.

8.6. WALL TWO: THE HASH FUNCTION DILEMMA

The obvious response to Wall One is: use `blake2b` then. It is a builtin, it is 1,500 times cheaper on-chain, problem solved.

It is not solved, because the hash appears on *both* sides. The Merkle path is recomputed on-chain during deposit *and* inside the circuit during withdrawal. And the two environments have exactly opposite preferences.

Hash (2-to-1)	On-chain (CPU)	In-circuit
Poseidon	600,802,860	≈ 240 constraints
<code>sha2_256</code>	581,923	$\approx 30,000$ constraints
<code>blake2b_256</code>	398,720	not in circomlib

The hash functions Cardano computes cheaply are the ones circuits prove expensively, and vice versa. Poseidon is 125 times cheaper in-circuit and 1,500 times more expensive on-chain. There is no hash function that is cheap on both sides of a Cardano ZK application today.

Following the SHA-256 Branch Honestly

Suppose we accept the trade and build the tree with `sha2_256`. What happens? Something initially encouraging, thanks to a fact established in Chapter 7:

Groth16 verification on Cardano costs 2,005,373,690 CPU steps *regardless of the circuit's size*. A 600,000-constraint circuit verifies for exactly the same price as our 216-constraint password circuit.

So on-chain, both sides now fit comfortably:

Transaction	CPU used	% of budget
Deposit: $20 \times \text{sha2_256}$	11,638,458	0.12%
Withdraw: 1 Groth16 verify + mint	$\approx 2,050,000,000$	20.5%

The on-chain problem disappears. The cost has not gone away, though; it has moved to the prover, and there it becomes severe:

Circuit	Poseidon tree	SHA-256 tree
Constraints (depth 20)	$\approx 4,800$	$\approx 600,000$
Powers of Tau needed	2^{13}	2^{20}
Proving key size	$\approx 2 \text{ MB}$	multiple GB
Browser proving	under a second	not viable

A 600,000-constraint Groth16 circuit needs a 2^{20} ceremony, a proving key measured in gigabytes, and tens of seconds to minutes of proving on a laptop. Nobody is downloading that into a browser tab.

And so the failure closes into a circle. Recall the three prover architectures from Section 7.4. Client-side proving is the only one that delivers privacy against the operator. A gigabyte proving key forces you to Architecture B, a server-side prover. But the server-side prover receives the `nullifier` and `secret`, which is the entire note.

A mixer whose proofs are generated by a server is not a mixer. The operator can link every deposit to every withdrawal, which is precisely the linkage the construction exists to break. Choosing an on-chain-cheap hash forces server-side proving, and server-side proving destroys the privacy property that justified the mixer.

That is Wall Two, and it is a genuine dilemma rather than an engineering task. Note that it would evaporate the moment Cardano gained a Poseidon builtin: the tree would be Poseidon on both sides, the circuit would be 4,800 constraints, and the proof would be generated in a browser in under a second.

8.7. WALL THREE: EUTXO CONTENTION

Suppose Wall One and Wall Two both vanish tomorrow: Cardano ships a Poseidon builtin, the tree costs almost nothing on-chain, the circuit is small, proving happens in the browser. The mixer still does not work, for a reason that has nothing to do with cryptography.

The Merkle Root Is Single-Threaded State

The pool's Merkle root lives in the datum of one UTxO. Every deposit must consume that UTxO and produce its successor. Two depositors who build transactions against the same pool UTxO produce two transactions spending the same input, and the ledger will accept exactly one. The loser's transaction is not delayed; it is invalid, and must be rebuilt against the new root and resubmitted.

Property	Ethereum	Cardano
Tree state	Contract storage	One UTxO's datum
Concurrent deposits	Ordered by the miner	All but one invalid
Deposits per block	Many	One
Block time	≈ 12 s	≈ 20 s
Max deposit rate	Hundreds/min	3/min

Three deposits per minute is the theoretical ceiling, achieved only if depositors coordinate perfectly. Under real contention (independent users, no coordination) the effective rate is far lower, and users experience it as transactions that simply fail.

Why This Is Fatal Rather Than Annoying

For most contended Cardano applications, this is a UX problem with known mitigations. For a mixer it is fatal, because **the anonymity set *is* the throughput**. Privacy in a shielded pool comes from the number of plausible alternative depositors. A pool that accepts three deposits a minute at absolute best, and realistically far fewer, cannot accumulate a meaningful anonymity set. The construction's security parameter and the platform's bottleneck are the same quantity.

And the Standard Mitigations Do Not Apply

- **Batching.** A coordinator collects deposits and inserts them together. This works technically, and the coordinator learns exactly which commitments arrived from which addresses in which batch. The batcher becomes the deanonymiser.
- **Sharding into N parallel pools.** Standard practice for Cardano concurrency, and here it divides the anonymity set by N . Ten pools give ten times the throughput and one tenth of the privacy. Anonymity does not shard.
- **Deferred insertion.** Deposits queue in their own UTxOs and are folded into the tree later. This works, but the queue UTxOs are public and sit there linking depositor addresses to commitments until they are absorbed, and the queue is itself a contended resource.
- **Hydra.** A Hydra head has no contention, and also a fixed, known participant set, which is the opposite of an anonymity set.

Every standard fix for eUTxO contention works by introducing a coordinator or by partitioning users. A mixer's security property is destroyed by coordinators and by partitions. Wall Three is not a performance limitation; it is a structural incompatibility between what makes eUTxO good (deterministic, local validation with no shared mutable state) and what a shielded pool needs (high-throughput shared mutable state).

8.8. THE SMALLER OBSTACLES

For completeness, several further problems would need solving even if the three walls came down.

Groth16's Per-Circuit Trusted Setup

Groth16 requires a fresh Phase 2 ceremony for every circuit. Since the tree depth is compiled into the circuit and each denomination is its own pool, every (denomination, depth) pair needs its own multi-party ceremony, its own proving key, and its own verification key. Five denominations means five ceremonies, and each one is a trust assumption a user must evaluate.

This is the strongest practical argument for **PLONK** over Groth16. PLONK uses a single *universal, updatable* setup: one ceremony serves every circuit up to a size bound, and it can be extended indefinitely by new contributors. For an application family like this, that difference is decisive.

The obstacle is that no production-grade PLONK verifier exists in Aiken today. Groth16 verification is three point decompressions and four pairings, which maps neatly onto Cardano's builtins. A PLONK verifier needs polynomial commitment openings, a transcript hash, and considerably more field arithmetic, and we have just measured what field arithmetic costs in a Plutus script. The same interpreter overhead that blocks on-chain Poseidon also stands between Cardano and a practical PLONK verifier. We return to this in Chapter 9.

Four Proofs Per Transaction

At 2.005 billion CPU steps per verification against a 10 billion budget, a transaction can carry at most four Groth16 verifications. Batched withdrawals (an obvious efficiency measure, and a privacy measure, since batching withdrawals blurs timing) cap out at four.

No Relay Market

Tornado's relay network exists because relaying is profitable at Ethereum fee levels. Cardano transaction fees are a fraction of an ADA, so the fee a relay could extract is small in absolute terms while the operational and legal risk is not. Nothing prevents relayers from existing on Cardano; economics makes them unlikely to.

Datum Growth and Sizes

The pool datum holds 20 filled-node hashes plus bookkeeping, under a kilobyte, comfortably within the 16,384-byte transaction limit. The withdrawal redeemer carries a 192-byte proof. Sizes are not a binding constraint here, which is worth saying explicitly: the limits that bite are execution units and contention, not bytes.

8.9. VERDICT

Component	Status on Cardano today
Withdrawal circuit	Works. Standard Circom over BLS12-381.
On-chain proof verification	Works. 2.0B CPU, ≈ 0.15 ADA, constant in circuit size.
Nullifier double-spend check	Works, and is <i>better</i> than Ethereum's: enforced by the ledger via token uniqueness.
Fixed denominations	Works. Trivial in eUTxO.
Recipient/fee binding	Works. Public inputs the validator checks.
On-chain Merkle insertion with Poseidon	Fails. 120% of CPU and 173% of memory budget at depth 20.
Merkle insertion with a builtin hash	On-chain fine; forces a 600k-constraint circuit, hence server-side proving, hence no privacy.
Concurrent deposits	Fails structurally. One deposit per block per pool; anonymity set equals throughput.

Tornado Cash cannot be built on Cardano today. Not because Cardano cannot verify zero-knowledge proofs (it verifies them cheaply and elegantly), but because it cannot maintain ZK-friendly cryptographic state on-chain, and because a shielded pool needs high-throughput shared mutable state that the eUTxO model is deliberately designed not to provide.

The distinction is worth stating precisely, because it generalises well beyond mixers:

Cardano today supports ZK applications where the **prover's statement is about data the verifier already has**, and struggles with applications where the statement is about **an accumulator the chain must maintain**.

Our password lock in Chapter 6 is the first kind: the hash sits in the datum, the validator reads it, done. Janus Wallet's authentication is the first kind. A mixer, a private rollup, an anonymous voting register with on-chain membership updates; all the second kind.

8.10. WHAT WOULD HAVE TO CHANGE

Change	Effect
A Poseidon builtin	Removes Walls One and Two entirely. Deposits become as cheap as blake2b, circuits stay at 4,800 constraints, proving returns to the browser. <i>This single primitive is the highest-leverage change available.</i>
Native scalar field arithmetic builtins	Removes the $\approx 2 \times 10^5$ -steps-per-operation floor. Makes on-chain Poseidon, PLONK verifiers, and custom curve work viable. Partially addressed by CIP-109's <code>modularExponentiation</code> .
A production PLONK verifier in Aiken	One universal ceremony instead of one per pool. Blocked today by the same field-arithmetic cost.
Larger execution budgets	Helps at the margin only. The memory budget rose from 11.25M to 14M to 16.5M, roughly 47% in total. On-chain Poseidon at depth 20 needs about $17\times$ the current memory budget. Incremental parameter increases will not close a gap of that size.
Contention-free shared state	The genuinely hard one. Requires either a coordinator (which breaks anonymity) or a different execution model.

The last row is why Midnight exists. Cardano's answer to applications needing private shared state is not to reshape the eUTxO ledger but to put them on a partner chain designed for it, with selective disclosure built in, which is also a deliberate answer to the regulatory question that opened this chapter. Chapter 9 takes that up.

8.11. CHAPTER SUMMARY

- Tornado Cash is three ingredients: a commitment Merkle tree, a nullifier set, and fixed denominations. The anonymity set is the pool's throughput.
- The withdrawal side ports cleanly to Cardano. The circuit is standard Circom, and verification costs 2.0B CPU steps regardless of circuit size.
- The nullifier set ports *better* than on Ethereum: minting a token named after the nullifier hash makes double-spend prevention a ledger invariant rather than a script check.
- **Wall One:** no Poseidon builtin. Measured, the cost floor for any elementary operation in a Plutus script is $\approx 2 \times 10^5$ CPU steps and ≈ 600 memory units, driven by interpreter overhead, not operand width, so no optimisation escapes it. A depth-20 Poseidon insertion needs 120% of the CPU and 173% of the memory budget.
- **Wall Two:** the hashes Cardano computes cheaply cost $\approx 30,000$ constraints in-circuit; the hash that costs ≈ 240 constraints cannot be computed on-chain. Choosing the on-chain-cheap hash forces a 600k-constraint circuit, hence server-side proving, hence no privacy against the operator.

- **Wall Three:** the Merkle root is one UTxO. One deposit per block per pool, ≈ 3 /minute at best. Every standard mitigation introduces a coordinator or partitions users, and both destroy anonymity.
- The general lesson: Cardano supports ZK where the statement concerns data the chain already holds, and struggles where the chain must maintain a ZK-friendly accumulator.
- A single Poseidon builtin would remove two of the three walls.

9. FINAL THOUGHTS AND FUTURE DIRECTIONS

9.1. WHERE WE STARTED, WHERE WE ARE

This book began with a cave, a magic door, and Ali Baba. It ends with execution unit measurements. That arc is deliberate: zero-knowledge proofs are one of the few areas of computer science where the elegant intuition and the engineering reality are separated by a very long distance, and a book that stops at the intuition leaves you unable to build anything.

Along the way we established, by construction and measurement rather than assertion:

- A complete Groth16 verifier compiles to **570 bytes** of Plutus Core (Chapter 7).
- Verifying a proof costs **2,005,373,690 CPU steps** and **34,219 memory units**, 20% of a transaction's CPU budget, about **0.147 ADA**, and that figure is **constant in the size of the circuit**.
- The cost floor for any elementary operation inside a Plutus script is roughly 2×10^5 **CPU steps** and **600 memory units**, driven by interpreter overhead rather than operand width (Chapter 8).
- Those two facts together explain almost everything about what is and is not possible.

Cardano is very good at verifying zero-knowledge proofs and very bad at computing anything ZK-adjacent on-chain. Every conclusion in this chapter is a corollary of that asymmetry.

9.2. THE CENTRAL ASYMMETRY

It is worth spelling out why the asymmetry exists, because it is not an accident or an oversight.

Cardano's BLS12-381 support is implemented as **builtins**: single Plutus Core operations backed by optimised native code, with a fixed price in the cost model. The miller-loop builtin does an enormous amount of arithmetic and is charged as one operation. That is why a pairing check is cheap.

Field arithmetic, by contrast, has no builtin. Aiken's **Scalar** is ordinary Aiken code over ordinary integers, and every multiply-and-reduce is several reduction steps in the CEK machine, each individually metered. That is why 1,413 field operations (one Poseidon hash) cost three hundred times more than four pairings.

Operation	CPU steps	Implementation
4 pairings + 5 decompressions (Groth16)	2,005,373,690	builtins
1 Poseidon hash ($t = 3$, estimated)	600,802,860	Aiken code
1 blake2b_256	398,720	builtin
1 field multiply-and-reduce	430,929	Aiken code
1 small integer addition	197,240	Aiken code

Read that table as a design principle rather than a benchmark. **On Cardano, cryptography that has been given a builtin is nearly free, and cryptography that has not been given a builtin is effectively unavailable.** There is no middle ground, because the interpreter overhead per operation is enormous relative to the arithmetic it performs. Optimising your Aiken does not help; the only thing that helps is a builtin.

This also explains why the incremental protocol-parameter increases do not change the picture. The transaction memory budget has gone from 11,250,000 to 14,000,000 to the current 16,500,000, about 47% in total. A depth-20 on-chain Poseidon insertion needs roughly 17 times the current budget. Parameter governance moves in tens of percent; the gap is in orders of magnitude.

9.3. THE MISSING PRIMITIVES

If builtins are the only lever, the natural question is which ones are missing. In rough order of impact:

1. A ZK-Friendly Hash Builtin (Poseidon or Poseidon2)

This is the single highest-leverage change available to Cardano's ZK ecosystem, and Chapter 8 quantified why. A Poseidon builtin would:

- make on-chain Merkle tree maintenance cost roughly what blake2b_256 costs today, a factor of $\approx 1,500$ improvement;
- keep circuits at ≈ 240 constraints per tree level instead of $\approx 30,000$, which keeps proving keys in megabytes and proving in the browser;
- and therefore let a whole class of accumulator-based applications run with client-side proving and real privacy: private voting, anonymous credentials, membership sets, shielded pools.

One builtin removes two of the three walls in Chapter 8. If you have influence over Cardano's roadmap and you care about zero-knowledge, this is the thing to argue for. Ethereum is having the same debate about Poseidon for the same reasons.

Note the appropriate caution: Poseidon is young by cryptographic standards, and arithmetisation-friendly hashes have a shorter cryptanalytic track record than SHA-2 or Keccak. Committing it to the ledger's cost model is a long-term commitment. Poseidon2 and its successors exist partly because the design space is still moving. This is a real argument for care, not for inaction.

2. Native Field Arithmetic

CIP-109 proposes `modularExponentiation`, which computes modular powers and (via negative exponents) multiplicative inverses. The CIP’s own motivation notes that computing an inverse currently consumes between 5% and 9% of the mainnet CPU budget. It remains at *Proposed* status.

That single builtin would help, but the broader need is a family of scalar field operations priced as builtins. That is what would remove the 2×10^5 -steps-per-operation floor and, with it, the reason a PLONK verifier is impractical.

3. A Second Proving System

Today Groth16 is the only fully supported option, and it comes with a circuit-specific trusted setup. The alternatives and their obstacles:

System	Status on Cardano
Groth16	Works today. Constant 2.0B CPU verification, 192-byte proofs. One ceremony per circuit.
PLONK	One universal, updatable ceremony for all circuits, a decisive advantage for any application family. Verifier needs polynomial commitment openings and substantial field arithmetic, which is exactly what Cardano prices badly. No production Aiken implementation.
Halo2	No trusted setup at all, and recursion-friendly. Verifier is heavier still. Not viable today.
STARKs	No trusted setup, hash-based rather than pairing-based. Verification is thousands of hash invocations, which Cardano <i>does</i> have builtins for, making this less obviously hopeless than it first appears, though proof sizes strain the 16,384-byte transaction limit.

The STARK row deserves a second look from anyone hunting for a research project. Cardano’s cost profile (cheap builtin hashes, ruinously expensive field arithmetic) is close to the opposite of what a pairing-based verifier wants and closer to what a FRI-based verifier wants. The binding constraint moves from execution units to transaction size. Whether a Cardano-suitable STARK verifier is possible is, as far as I know, an open and unexplored question.

9.4. THE ECOSYSTEM, HONESTLY ASSESSED

Cardano's ZK ecosystem in 2026 is small, real, and roughly five years behind Ethereum's. Pretending otherwise does newcomers no favours; so does pretending it is empty.

Project	What it is
Janus Wallet	The most complete open-source ZK application on Cardano. Password authentication via Groth16 with no seed phrase. Server-side proving, Poseidon, BLS12-381, Aiken validators. The reference for how a real ZK product is assembled.
aiken-zk (Eryx)	Language extension and CLI that generates circuits, verifiers, and MeshJS redeemers from <i>offchain</i> annotations. Catalyst Fund 13. The most promising developer-experience work in the space, and its mocked Merkle hash is an honest signpost to the platform's real gap.
Modulo-P (ak-381, Cardano-Semaphore)	A reusable Aiken Groth16 verifier and a port of Semaphore, the anonymous-signalling protocol. Semaphore is the same Merkle-tree-plus-nullifier shape as Chapter 8, which makes it the natural place to watch for progress on the accumulator problem.
cardano-foundation/bls	A "gym" of BLS12-381 examples in Aiken, from the Foundation. Where to look for primitive-level patterns.
Midnight	Cardano's ZK partner chain. Federated mainnet launched March 2026. Dual-ledger architecture, selective disclosure, separate token model.

Two observations. First, almost every project on that list uses Groth16 over BLS12-381 with Circom and snarkjs, which is why this book taught that stack: it is not one option among many, it is the ecosystem. Second, the list is short enough that a competent developer can read all of it in a week, which is either intimidating or an opportunity, depending on temperament.

9.5. WHAT ACTUALLY WORKS TODAY

Chapter 8 was a catalogue of failure, so let us be precise about the complement. The test is the one from Section 8.9: does the proof concern data the chain already holds, or state the chain must maintain?

Authentication and Access Control

Prove knowledge of a credential without revealing it. The commitment sits in a datum; the validator reads it. This is Chapter 6, and Janus Wallet is the production version. **Works today, entirely.**

Off-Chain Computation Compression

Move an expensive computation off-chain and prove it was performed correctly. Because verification is constant at 2.0B CPU steps regardless of circuit size, this trade is spectacular: an arbitrarily large computation, verified for 0.147 ADA. This is precisely what aiken-zk's offchain blocks are for. **Works today, and is underexploited.**

If you are looking for a ZK project on Cardano with a favourable effort-to-value ratio today, this is it. The constant verification cost means the more expensive your computation, the better the trade looks, and no accumulator is involved, so none of Chapter 8's walls apply.

Selective Disclosure and Attestations

Prove you are over 18, or resident in a jurisdiction, or accredited, without revealing the underlying document. The issuer's public key or credential root lives in a datum or a reference input; the chain maintains nothing. **Works today.**

Private Voting with Off-Chain Roll-Up

Prove membership in an eligible-voter set and cast a vote nobody can link to you. This works if the membership root is fixed at the start of the vote: published once, then read-only. It hits Chapter 8's walls only if the register must be updated on-chain during the vote. **Works today, with that design constraint.**

Cross-Chain and Light-Client Proofs

Prove that a block header chain or state transition on another chain is valid. Verification is one constant-cost check. The interesting nuance is that the circuit must implement the *other* chain's hash function, which is usually Keccak or SHA-256, expensive in-circuit, exactly the Wall Two trade-off, but here there is no on-chain accumulator forcing the issue. **Works today, at the cost of large circuits.**

The Pattern

Works today	Blocked today
Statement about a fixed public value in a datum	Statement about an accumulator the chain must update
One proof, one transaction, no shared state	Many users contending for one state UTxO
Root published once, read many times	Root recomputed on-chain per transaction
Circuit-expensive hashes acceptable	Requires a hash cheap both on-chain and in-circuit

9.6. LAYER 2 AND THE MIDNIGHT ANSWER

Cardano’s institutional answer to the applications in the right-hand column is not to reshape the eUTxO ledger. It is to move them.

Hydra removes contention by moving state into a head with a fixed participant set. That is genuinely useful for throughput and useless for anonymity (a known participant set is the opposite of an anonymity set), but it is a strong fit for ZK applications where the parties are known and the *data* is what needs hiding.

Midnight is the more direct answer: a partner chain designed from the start for ZK smart contracts, with a dual-ledger architecture separating public from private data, and selective disclosure as a first-class feature rather than a bolt-on. Its federated mainnet launched in March 2026.

The strategic logic is coherent, and worth understanding rather than merely noting. Cardano’s L1 optimises for deterministic, locally verifiable, cheaply priced validation. Those properties are why the eUTxO model has no shared mutable state, and the absence of shared mutable state is exactly Wall Three. You cannot have both. So the L1 keeps its guarantees and hosts ZK *verification*, and a purpose-built chain hosts ZK *state*.

Be alert to the trade this makes on your behalf. Midnight’s selective disclosure is a deliberate design position: privacy that can be lifted under defined conditions. That is a genuine answer to the regulatory problem that opened Chapter 8, and it is also, unavoidably, weaker than unconditional privacy. Whether that is the right trade depends on the application and on who you think should hold the key. It is not a technical question, and it should not be presented as one.

9.7. ETHICS, REGULATION, AND BUILDING ANYWAY

This book contains a chapter analysing a sanctioned protocol. It seems worth being direct about how to think about that.

Zero-knowledge proofs are unusually dual-use. The same construction that lets a refugee prove eligibility for aid without revealing their identity lets someone launder proceeds of crime. The mathematics does not distinguish. Neither, in the Tornado Cash case, did the enforcement action: OFAC sanctioned the smart contract addresses themselves, and a developer was prosecuted over software he did not control at the time of the conduct at issue. Those actions remain contested, and a fuller reckoning belongs to lawyers and courts rather than to this book.

What belongs here are three practical positions.

Privacy is the default in every other part of life, and its absence on public blockchains is an artefact rather than a virtue. A blockchain where every salary payment, medical invoice, and political donation is permanently public to everyone is not a neutral design; it is a surveillance system that happens to be decentralised. Work on privacy needs no special justification.

Threat modelling is part of engineering. Chapter 7's point about server-side proving is the concrete form of this: a system marketed as zero-knowledge whose prover runs on someone else's server provides no privacy against that someone. If you build ZK systems, state clearly what is hidden, from whom, and under what assumptions. Users cannot evaluate a claim like "uses zero-knowledge proofs".

Selective disclosure is a legitimate engineering answer, not a capitulation. Systems where the user can prove specific facts to specific counterparties (rather than either revealing everything or nothing) cover most real requirements, and are the direction Cardano and Midnight have chosen. Absolute anonymity is a narrower need than the discourse suggests.

The interesting question is not "should privacy exist" but "who holds the disclosure key, and under what process". That is a governance question. Building systems that make it an explicit, auditable governance question, rather than an implicit property of whoever runs the server, is a contribution.

9.8. WHAT TO BUILD NEXT

Concrete suggestions, ordered by ratio of value to difficulty, for anyone who finished this book and wants to contribute.

1. **Add replay protection to the Chapter 6 contract.** Add a nonce to the datum and the circuit's public inputs, require the continuing output to increment it. This is the single most important missing feature, it is a genuine exercise, and it takes an afternoon.
2. **Implement Poseidon in Aiken and publish the measurements.** Yes, it will be too expensive to use; that is the point. Chapter 8's figures are estimates extrapolated from measured primitives. A real implementation with real numbers would turn them into data, and data is what a builtin proposal needs.
3. **Build something using off-chain computation compression.** Take an expensive on-chain computation, move it into a circuit, verify the proof for a constant 0.147 ADA. No accumulator, no contention, no walls. The best available ratio of effort to result on Cardano today.
4. **Write the missing tooling.** A library that generates Aiken types and MeshJS

encoders from a `verification_key.json` would eliminate most of the failure modes in Section 7.10. The bugs in this book’s tables are encoding bugs, and encoding bugs are what code generation exists to prevent.

5. **Investigate a STARK verifier for Aiken.** Speculative and possibly a dead end. Also the one item on this list where Cardano’s peculiar cost profile (cheap builtin hashes, expensive field arithmetic) might turn out to be an advantage rather than a handicap.
6. **Argue for a Poseidon builtin, with numbers.** The measurements in Chapter 8 exist to be reused. Cardano’s governance process responds to concrete cases.

9.9. WHERE TO CONTINUE

Resource	Why
aiken-lang.org	Language reference and the <code>stdlib</code> <code>bls12_381</code> modules. Read the <code>scalar</code> source; it explains a great deal.
docs.circom.io	The circuit language, and the underlying R1CS model.
github.com/iden3/circomlib	The primitive library. Read <code>poseidon.circom</code> against Chapter 8’s round counts.
github.com/leobel/janus-wallet	A complete ZK product. Read the circuit and the Aiken validators together.
github.com/eryxcoop/cardano-zk-aiken	Where Cardano’s ZK developer experience is being built.
cips.cardano.org	CIP-101 (keccak256), CIP-109 (modular exponentiation), CIP-127 (ripemd-160). How primitives actually arrive.
zkproof.org	Standards work, and a reading path into the underlying literature.

And the two papers worth reading in full rather than summarised: Goldwasser, Micali and Rackoff’s 1985 paper for what a zero-knowledge proof *is*, and Groth’s 2016 paper for the construction whose verification equation we implemented in Chapter 6.

9.10. CLOSING

This book set out to make zero-knowledge proofs on Cardano buildable rather than merely admirable. Nine chapters later, the honest summary is two sentences.

Cardano can verify a zero-knowledge proof about any computation, of any size, for about 0.147 ADA and 570 bytes of on-chain code. It cannot yet maintain the kind of cryptographic state that the most ambitious ZK applications need, and the reason is a missing hash builtin and a ledger model that deliberately has no shared mutable state.

The first sentence is a remarkable capability that very few developers on Cardano are currently using. The second is a real limitation with a known, tractable, single-item fix at the top of its list.

Neither sentence was findable in one place when this project started, which is why the project existed. Both are now measured, reproducible from the repository, and free to reuse.

Everything in this book (the LaTeX source, the Circom circuits, the Aiken validators, the benchmarks, the measurements) is open source under AFL-3.0. Corrections are more welcome than praise; the numbers in Chapter 8 in particular are extrapolations that deserve to be replaced by someone's real implementation. Go build something. Publish what breaks.

10. SITOGRAPHY: THE ZK ON CARDANO LANDSCAPE

10.1. HOW TO USE THIS CHAPTER

Every project, tool, specification and paper referenced in this book is collected here with a clickable link. If you are reading the PDF, the titles are live: click them.

The list is deliberately opinionated. Where two resources cover the same ground, the more useful one is listed and the other is not. Where a project is promising but not yet usable, that is stated.

Links rot. This sitography was verified in July 2026. The canonical, updatable version lives in the repository, and a dead link is a legitimate issue to open: github.com/elRaulito/ZK-from-zero-on-Cardano

10.2. THIS BOOK

ZK from Zero on Cardano The repository. LaTeX source, the zk-password reference implementation, the benchmarks behind Chapter 8, and the rendered PDFs on the rendered branch.

Catalyst proposal (Fund 14, #1400129) The original proposal, milestone reports, and close-out documentation.

raul.it The author.

10.3. CARDANO ZK PROJECTS

This is the complete set of open-source ZK work on Cardano known to the author at the time of writing. It is short enough to read in a week. That is either intimidating or an opportunity.

Applications

Janus Wallet The most complete open-source ZK application on Cardano. A wallet where the user authenticates with a password rather than a seed phrase, proving knowledge of it via a Groth16 circuit. Circom, Poseidon, BLS12-381, Aiken validators, and a Node.js proving service. **Read the circuit and the validators together**; it is the best available worked example of how a real ZK product is assembled. Discussed in Chapters 5, 6 and 7.

Cardano-Semaphore A port of Semaphore, the anonymous-signalling and group-membership protocol, by Modulo-P. Semaphore has the same Merkle-tree-plus-nullifier shape as the mixer analysed in Chapter 8, which makes this the project to watch for progress on the on-chain accumulator problem.

Midnight Cardano's ZK partner chain. Dual-ledger architecture separating public from private data, with selective disclosure as a first-class feature. Federated

mainnet launched March 2026. Cardano’s institutional answer to the applications Chapter 8 shows the L1 cannot host.

Libraries and Tooling

- aiken-zk (Eryx)** A language extension and CLI that generates the Circom circuit, the Aiken verifier, and even the MeshJS redeemer from an `offchain` annotation in your validator. Catalyst Fund 13. The most promising developer-experience work in the space. Note the honest caveat on its Merkle tree hash, discussed in Section 7.8.
- ak-381 (Modulo-P)** A reusable Aiken implementation of a Groth16 verifier over BLS12-381. If you would rather import a verifier than write the one in Chapter 6, start here.
- cardano-foundation/bls** A “gym” of BLS12-381 examples in Aiken, from the Cardano Foundation. The place to look for primitive-level patterns.
- poseidon-bls12381** A TypeScript Poseidon implementation targeting BLS12-381. Useful off-chain, and a solution to the `circomlibjs` field-mismatch trap described in Section 7.2.

Background Reading

- Aiken’s BLS12-381 Primitives Explained** Cardano Foundation walkthrough of what the pairing builtins make possible.
- Unlocking More Opportunities with Plutus V3** The IOG post introducing the primitives this entire book depends on.

10.4. CIRCUITS AND PROVING

- Circom documentation** The circuit language. Start with *Getting Started*, then the *Circom Language* reference. Remember `--prime bls12381`.
- circomlib** The primitive circuit library: Poseidon, comparators, bit decomposition, Merkle proofs. Read `poseidon.circom` against the round counts in Section 8.5.
- snarkjs** Trusted setup, proving, and verification. The reference implementation and the one this book uses throughout.
- rapidsnark** A C++ prover, 5 to 10 times faster than snarkjs, reading the same `.zkey` files. Check its BLS12-381 support before committing.
- ark-circom** Drive Circom circuits from Rust via arkworks. First-class BLS12-381 support.
- gnark** A Go proving library with its own circuit DSL. No Cardano verifier, but good for understanding proving systems from the inside.
- Noir** A far nicer ZK language than Circom. Listed so you know it exists and know why this book does not use it: its backends have no Aiken verifier.

10.5. CARDANO DEVELOPMENT

- Aiken** The on-chain language. The **standard library** docs are where the `bls12_381` modules live, and reading the `scalar` source explains a great deal about why

Chapter 8 reaches the conclusion it does.

MeshJS The TypeScript SDK used for the transactions in Chapter 7. See **Smart Contract Interactions** for the `MeshTxBuilder` patterns.

Lucid Evolution / Evolution SDK The main alternative to MeshJS. Same job, different ergonomics. Originally from Anastasia Labs, now maintained by No Witness Labs and being renamed to Evolution SDK, so expect the package names to move.

Blockfrost Hosted chain API. Free tier is sufficient for everything in this book.

Koios A public, no-key-required Cardano API. The live protocol parameters quoted in Chapters 7 and 8 came from here.

Maestro Another provider, with indexing features useful once you outgrow simple UTxO queries.

Cardanoscan and **cexplorer**. Explorers. Useful for inspecting datums and script execution units on a submitted transaction.

Protocol parameters guide What each parameter means. For live *values*, query Koios or a node.

10.6. CARDANO IMPROVEMENT PROPOSALS

How cryptographic primitives actually arrive on Cardano. Chapter 9 argues that a Poseidon builtin belongs on this list and is not yet on it.

CIP-381 The BLS12-381 primitives themselves. The foundation of every ZK contract on Cardano today.

CIP-101 Keccak-256, which enables Ethereum signature verification in scripts.

CIP-109 `modularExponentiation`, and with it on-chain modular inverses. Still at *Proposed* status. The most directly relevant open proposal to the cost floor measured in Section 8.5.

CIP-127 RIPEMD-160.

The CIP repository Where to propose one, and where to read the discussion on proposals that have not landed.

10.7. PAPERS AND FOUNDATIONS

The two worth reading in full rather than in summary are marked.

Goldwasser, Micali, Rackoff (1985) *The Knowledge Complexity of Interactive Proof Systems*. The paper that defines what a zero-knowledge proof *is*. **Read this one in full.**

Groth (2016) *On the Size of Pairing-Based Non-interactive Arguments*. The construction whose verification equation we implemented in Chapter 6. **Read this one too.**

Grassi et al. (2019) *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. The round structure costed in Section 8.5.

Gabizon, Williamson, Ciobotaru (2019) *PLONK*. The universal-setup construction that Chapter 9 argues Cardano should eventually support.

Ben-Sasson et al. (2018) *Scalable, Transparent, and Post-quantum Secure Computational Integrity*. STARKs. Relevant to the speculative research direction in Section 9.3.

ZKProof Standards work, plus a curated reading path into the wider literature.

The Halo2 Book For understanding what a no-trusted-setup proving system requires of its verifier.

10.8. REFERENCE IMPLEMENTATIONS WORTH READING

Not Cardano projects, but the canonical implementations of the ideas this book discusses. Reading them is how you learn what a mature ZK codebase looks like.

tornado-core The original mixer contracts and circuits reverse-engineered in Chapter 8. Listed for study. As that chapter sets out at length, the protocol was sanctioned in 2022 and this book does not recommend deploying anything derived from it.

Semaphore Anonymous signalling on Ethereum, and the upstream of Cardano-Semaphore. A cleaner introduction to Merkle-tree-plus-nullifier design than Tornado.

Zcash The first large-scale zk-SNARK deployment, discussed in Chapter 3.

Mina Recursive proof composition taken to its logical conclusion: a whole blockchain compressed into one constant-size proof.

10.9. WHERE TO ASK

Aiken Discord The most useful place for Aiken and on-chain questions, including the BLS12-381 builtins.

Cardano Forum Longer-form technical discussion. The *Smart Contracts* category is where execution-unit questions get answered.

Cardano Stack Exchange Best for specific, answerable questions.

Cardano Developers (Telegram) Fast, informal, good for unblocking.

ZK Hack Not Cardano-specific, and the best community for questions about the cryptography itself: study groups, puzzle challenges, and talks. The **Zero Knowledge podcast** is the single best way to keep up with the field.

If you build something with this book, the most useful thing you can do is write down what broke. The failure tables in Chapter 7 and the measurements in Chapter 8 exist because the problems were documented rather than merely worked around. This sitography is short. Help make it longer.